GPSDO v1.08 — El manual completo (de cero a LOCK)

English | Polski | **Español**

📄 PDF: English · Polski · Español

📖 Inicio del proyecto · README · Changelog · Tuner

Firmware: GPSDO v1.08.71rt de jmnlabs (sobre el GPSDO original de André Balsa, el lazo PI de Lars Walenius, el acumulador multinivel de Alan Cashin y el filtro de Kalman propio del autor) **Placa:** WeAct BlackPill STM32F411CE · **OCXO:** 10 MHz (p. ej. Vectron C4550) **GPS:** receptor u-blox en Serial1 (LEA-M8T / NEO-M8T / ZED-F9T o NEO-6M/7M)

Autor: Jarosław Marek Niewiński (jmnlabs) **Asistentes:** Claude Opus 5 (Anthropic) · GLM-5.3 Max (Z.ai) · Qwen3.8-Max — las versiones polaca y española de este manual son de GLM-5.3 Max Este manual no presupone **nada**. Si nunca ha compilado firmware, nunca ha grabado un microcontrolador y nunca ha usado un terminal serie, empiece en la Parte 1 y haga exactamente lo que ahí se dice, en orden. Cada paso indica qué escribir y qué debería ver. Nada importante queda como «ejercicio para el lector».

> **Resumen en 60 segundos:** instale Arduino IDE con el núcleo STM32 (no el 3.0.0), abra el sketch, elija su pantalla en `gpsdo_config.h`, compile con *Newlib Nano + Float printf*, grabe con ST-Link o DFU. **Nunca haga un borrado completo del chip (Erase Chip)** — sus ajustes y calibración viven en el sector 7 y solo un borrado completo puede destruirlos. Después, por serie a 115200: `CT`, espere, `LC`, espere, `LA 12`, `SAW 1`, `ES`. Mire cómo la fase se mantiene en cero. Listo.

Índice

- Parte 1 — Compilación del firmware
- Parte 2 — Grabar el firmware, y la regla del sector 7
- Parte 3 — Primer arranque: calibración, algoritmo, guardado
- Parte 4 — Los catorce algoritmos (0–13) y sus parámetros
- Parte 5 — Pantallas: qué significa cada campo
- Parte 6 — Telemetría serie (el informe de 6 líneas)
- Parte 7 — Referencia de comandos (todos)
- Parte 8 — El sintonizador en el PC
- Parte 9 — Ajustes, el flash ring y el desgaste
- Parte 10 — Resolución de problemas
- Apéndice A — Cómo funciona un GPSDO, en palabras llanas
- Apéndice B — PID para reacios
- Apéndice C — Glosario
- Apéndice D — El filtro de Kalman en palabras llanas

Parte 1 — Compilación del firmware

1.1 Qué hay que instalar

1. **Arduino IDE** (1.8.x o 2.x — ambos sirven).
2. **El núcleo STM32duino, versión 2.2.0 a 2.12.0.** Boards Manager → busque «stm32» → **STM32 MCU based boards by STMicroelectronics**. Instale la 2.12.0 (la última soportada).

> **AVISO — el núcleo 3.0.0 no funciona.** Salió en julio de 2026 y rompe este proyecto por tres vías: `ltoa()` ya no existe (error de compilación), `HardwareSerial Serial2(PA3, PA2)` no compila, y TFT_eSPI deja de manejar el panel (pantalla blanca permanente). Quédese en la 2.12.0 o anterior hasta que el proyecto anuncie lo contrario.

1. **Librerías** (Library Manager, instale por nombre exacto):

Librería	Para qué
STM32duino FreeRTOS	el sistema operativo
TinyGPS++	análisis de frases GPS
U8g2	pantallas OLED (solo si activa una)
Adafruit AHTX0	sensor de temperatura/humedad AHT10/AHT20
Adafruit BMP280	sensor de presión/temperatura BMP280
Adafruit INA219	monitor de tensión/corriente de alimentación
hd44780	LCD de caracteres 20x4 (solo si lo activa)
TFT_eSPI	pantalla TFT por SPI

1. Nada más. No hay librería EEPROM — los ajustes viven en el flash del propio chip (Parte 2).

1.2 Abrir el sketch

Descomprima el proyecto. Abra `GPSDO_FreeRTOS/GPSDO_FreeRTOS.ino` en Arduino IDE. La carpeta debe conservar su nombre — Arduino exige que el nombre de la carpeta coincida con el del archivo `.ino`.

1.3 Elegir el hardware en `gpsdo_config.h`

Este único archivo decide qué se compila. Las líneas activas empiezan por `#define`; las apagadas, por `//`. Descomente exactamente lo que tiene su placa y comente lo que no. La configuración de fábrica es una placa completa (TFT grande + todos los sensores); adáptela a la realidad.

Pantalla principal — elija exactamente una:

Marca	Hardware
`GPSDO_TFT_ILI9488`	TFT SPI 320x480 (por defecto; trabaja en horizontal 480x320)
`GPSDO_TFT_ST7789`	TFT SPI 240x320
`GPSDO_TFT_ILI9341`	TFT SPI 240x320
`GPSDO_OLED_SH1106` / `_SSD1306` / `_SSD1309`	OLED 128x64 por I2C (solo uno)
`GPSDO_LCD_20x4`	HD44780 20x4 vía PCF8574 por I2C
`GPSDO_TM1637` / `GPSDO_TM1637_6`	reloj LED de 4 / 6 dígitos (incompatibles entre sí, y no junto al LCD)
`GPSDO_HT16K33`	reloj LED de 4 dígitos por I2C (activo por defecto; independiente de la pantalla principal)

Sensores y extras (todos activos de fábrica — comente los que no tenga):

Marca	Hardware
`GPSDO_AHT10`	AHT10/AHT20 en I2C
`GPSDO_BMP280_I2C`	BMP280 en I2C
`GPSDO_INA219`	INA219 en I2C
`GPSDO_LM75`	termómetro de la familia LM75 en I2C — LM75A/B, PCT2075, TMP75/175, TMP1075 — en `GPSDO_LM75_ADDR` (0x48); sin biblioteca. Su temperatura va tras `CPU:` en la línea de sensores del informe `RH` (build 68). Otro chip en 0x48 puede pasar por uno (un ADS1x15, un PCF8591): cambie entonces la dirección o deje fuera esta marca
`GPSDO_VCC` / `GPSDO_VDD`	medir el carril de 5 V / 3,3 V
`GPSDO_UBX_CONFIG`	poner el NEO-6M/7M en modo binario al arrancar
`GPSDO_FAKE_UBLOX`	**clon** chino de u-blox: solo sonda de baudrate, cero configuración UBX (véase la Parte 3.2)
`GPSDO_GPS_TIMING`	soporte de receptores de tiempo (LEA-6T/M8T/NEO-M8T/ZED-F9T): survey-in, qErr
`GPSDO_PICDIV`	salida de armado del divisor picDIV en PB3
`GPSDO_LTIC`	**el detector de fase por hardware** (rampa TIC en PA1) — necesario para los algoritmos 10, 11 y 12. Actívelo SOLO si el detector está físicamente montado: sin él PA1 queda flotante y el lazo disciplina el OCXO con ruido del ADC
`GPSDO_LTIC_ACTIVE_RESET`	descarga el condensador del detector por PA1 justo después de cada lectura (activada de serie; déjela activada con cualquiera de los dos detectores — el RC clásico de Kaashoek y la variante con fuente de corriente —, véase la Parte 3.3)
`GPSDO_PWM_DITHER`	tensión de control de 24 bits desde un PWM de 13 bits con dither (no lo apague — este es el buen DAC)
`GPSDO_DAC_EXT`	DAC AD5680 externo, 18 bits, bit-bang en PB4/PB0/PB2 — se puede compilar junto con `GPSDO_PWM_DITHER`; el comando `DAC` elige la ruta activa en tiempo de ejecución. Desactiva silenciosamente `GPSDO_TM1637`/`GPSDO_TM1637_6` y `GPSDO_GEN_2kHz_PB5` (PB4 es el chip-select del DAC)
`GPSDO_SPAN_SENSE`	el segundo polo del puente de span del EFC en **PB14** (se entrega ACTIVADO): una calibración `CT` por cada posición del span, conmutada cuando el puente se mueve — vea `SPAN` en la Parte 7. Una placa sin ese cable lee FULL con el pull-up interno y conserva su única calibración. Desactívelo solo si en su placa PB14 está conectado a otra cosa
`GPSDO_GEN_2kHz_PB5`	onda cuadrada de prueba de 2 kHz en PB5
`GPSDO_BLUETOOTH`	módulo HC-06 en PA2/PA3
`GPSDO_BLUETOOTH_PARALLEL`	USB **y** Bluetooth a la vez (activo por defecto)

El archivo se niega a compilar (a propósito) si elige dos pantallas del mismo tipo o dos elementos que se pelean por los mismos pines. Lea el mensaje `#error` — nombra el conflicto.

Deje activos `GPSDO_LTIC` y `GPSDO_PWM_DITHER`, salvo que de verdad no tenga el hardware del detector: sin LTIC pierde los algoritmos 10–13, que son todo el sentido de la v1.08.

1.4 Configurar TFT_eSPI (solo pantallas TFT)

TFT_eSPI se configura **en la librería**, no en el sketch. Busque `Arduino/libraries/TFT_eSPI/User_Setup.h` y póngale el bloque de su panel:

240x320 (ST7789 o ILI9341):

```
#define ST7789_DRIVER          // or ILI9341_DRIVER
#define TFT_WIDTH  240
#define TFT_HEIGHT 320
#define TFT_MISO PA6      // required on STM32 even if the display has no MISO pin
#define TFT_MOSI PA7
#define TFT_SCLK PA5
#define TFT_CS   PB13
#define TFT_DC   PB12
#define TFT_RST  PB15
#define TFT_RGB_ORDER TFT_BGR
#define TFT_INVERSION_OFF       // fixes inverted colours on some ST7789 modules
#define LOAD_GLCD               // font 1 — frequency readout + splash
#define LOAD_FONT2              // font 2 — header + data grid
#define LOAD_FONT4              // font 4 — status bar, busy messages
#define SPI_FREQUENCY 40000000  // 40 MHz works; drop to 27 MHz for long wires
```

320x480 (ILI9488 / ILI9486):

```
#define ILI9488_DRIVER          // works for both ILI9488 and ILI9486
#define TFT_WIDTH  320
#define TFT_HEIGHT 480
// mismas líneas TFT_MISO/MOSI/SCLK/CS/DC/RST/RGB_ORDER que arriba
#define LOAD_GLCD               // font 1 — splash credits
#define LOAD_FONT4              // font 4 — splash subtitle path
#define LOAD_GFXFF              // REQUIRED on this panel — free fonts
#define SPI_FREQUENCY 40000000  // don't skimp: this panel pushes 2.4x the pixels
```

Cableado del panel: SCK→PA5, SDI→PA7, RES→PB15, D/C→PB12, CS→PB13. `TFT_MISO PA6` debe estar definido aunque no haya nada conectado a él.

1.5 `build_opt.h` — no lo toque

El archivo contiene tres flags del compilador y no requiere ninguna acción:

```
-DSERIAL_RX_BUFFER_SIZE=256 -DSERIAL_TX_BUFFER_SIZE=512
-DCDC_TRANSMIT_QUEUE_BUFFER_PACKET_NUMBER=16
```

La primera línea agranda los búferes serie para que no se pierdan frases GPS. La segunda amplía la cola de transmisión USB-CDC de 128 bytes a 1 KB, de modo que el informe de telemetría de 1 Hz (unos 400 caracteres) siempre quepa — sin ella, un host que se conecta y no lee podía frenar el informe durante segundos. El núcleo STM32 recoge el archivo automáticamente. No lo borre.

1.6 El menú Tools — placa y librería de C

En Arduino IDE: **Tools** →

- **Board:** "Generic STM32F4 series" → **BlackPill F411CE**.
- **C Runtime Library: Newlib Nano + Float Printf/Scanf**. Obligatorio. Sin ello el firmware compila, pero imprime basura donde debería haber un número en coma flotante (todas las tensiones, frecuencias y fases).

1.7 Activar USB CDC (para tener consola por USB)

CDC es lo que convierte `Serial` en un puerto COM virtual por el cable USB. El rótulo de arranque, la línea de comandos y el informe de 1 Hz viajan por ahí. Sin CDC la placa sigue funcionando — la pantalla se actualiza, el lazo disciplina el oscilador — pero por USB parece completamente muerta: sin rótulo, sin prompt, nada.

1. **Tools → USB support (o "USB support (if available)") → "CDC (generic Serial supersedes U(S)ART)".** El nombre exacto varía algo entre versiones del núcleo; elija la que mencione **CDC** y **generic Serial**.
2. Recompile y regrabe. Cambiar este ajuste cambia el firmware — no es un conmutador en caliente.
3. Tras grabar, **pulse una vez el botón RESET de la placa**. El chip pasa de dispositivo USB DFU (grabación) a dispositivo CDC serie; sin el reset, algunos hosts siguen hablando con el dispositivo antiguo, ya desaparecido.
4. Aparece un puerto COM nuevo (Windows: "STM32 Virtual COM Port", **VID 0483, PID 5740**). Si Windows lo muestra como "Unknown device" o "device descriptor request failed", instale el driver una vez con Zadig — véase también la Parte 2.5.
5. Abra el puerto a 115200. El firmware **espera hasta 3 segundos** tras el reset a que el host abra el puerto antes de imprimir el rótulo — un driver lento no se traga las primeras líneas. Si abre el terminal y no ve nada, pulse RESET otra vez con el terminal ya abierto.

Dos cosas que conviene saber:

- El modo DFU (grabación) y el modo CDC (trabajo) son para Windows **dos dispositivos USB distintos**. Los primeros ciclos de grabación pueden disparar cada uno su ronda de instalación de drivers — es normal, se asienta.
- Con `GPSDO_BLUETOOTH_PARALLEL` compilado (la configuración de fábrica), todo lo que sale por USB se duplica al UART Bluetooth a **57600** — una consola de reserva lista, por si el USB se resiste. El informe de cada segundo va a cada puerto por separado (build 68), así que con un host USB conectado que no lee como mucho se pierde la copia del informe en el USB — nunca la del Bluetooth — y la placa nunca se bloquea.

1.8 Compilar y comprobar el tamaño

Sketch → Verify/Compile. Al terminar, lea la línea `Sketch uses NNNNNN bytes`.

> **NNNNNN debe mantenerse por debajo de 393216.** Son 384 KB — todo lo que hay por encima pertenece al anillo de ajustes del sector 7 (Parte 2). Ignore el porcentaje que imprime el IDE: se calcula contra los 512 KB completos del chip, así que una compilación que «cabe» en porcentaje puede invadir el sector 7. La propia v1.08 ocupa unos 300 KB (build 71: 303 364 bytes) — margen de sobra, pero no ignore un salto repentino.
>
> **¿Qué binario está corriendo realmente?** El banner imprime la hora de compilación — y esa hora pertenece al **sketch**, no al firmware. El compilador de Arduino no rehace una unidad cuyas fuentes no han cambiado, así que editar `GPSDO_algorithms.cpp` y subir deja intacto el objeto del sketch con una fecha anterior dentro. El banner imprime además `image CRC32`, calculado al arrancar desde la propia flash: no puede salir de una caché de compilación, cambia si cambia un solo byte de cualquier unidad, y dos placas con el mismo binario imprimen el mismo número. Cuando un registro y un recuerdo no coinciden, ése es el que vale. `V` lo vuelve a imprimir en cualquier momento.
>
> Si quiere que la marca de tiempo también sea correcta, ejecute `python3 tools/bumpbuild.py` antes de compilar — toca `build_id.h`, que el sketch incluye sólo por eso, y con ello el compilador rehace el sketch y nada más. El número aparece en el banner como `build N`.

Parte 2 — Grabar el firmware, y la regla del sector 7

2.1 Cómo se divide el flash de 512 KB

Sectores	Rango de direcciones	Contenido
0 – 5	0x08000000 – 0x0803FFFF	**Su firmware** (384 KB)
6	0x08040000 – 0x0805FFFF	reserva (margen de crecimiento)
7	**0x08060000 – 0x0807FFFF**	**el flash ring: ajustes + calibración + datos aprendidos**

En el sector 7 vive todo lo que el aparato ha aprendido de sí mismo:

- sus ajustes guardados (algoritmo elegido, todos los parámetros del lazo, zona horaria),
- la calibración de sensibilidad del oscilador hecha por `CT`,
- la calibración del detector de fase hecha por `LC`,
- el modelo aprendido de deriva/amortiguación y el último punto de trabajo.

Reconstruir todo eso cuesta una hora de banco. Proteger el sector 7 es por tanto **la regla operativa más importante de todo el proyecto**.

2.2 La regla de oro

> **Una grabación normal nunca toca el sector 7. El borrado completo del chip lo destruye.**
>
> - La subida desde Arduino IDE, DFU y el bootloader STM32duino reprograman solo los sectores 0–5. Sus ajustes sobreviven cada actualización rutinaria.
> - «Erase Chip» en ST-LINK Utility / STM32CubeProgrammer, o `erase` sin rango de direcciones en J-Link, borra el chip entero **incluido el sector 7**. Nunca lo use en una unidad funcionando y calibrada.

Si graba a mano con ST-Link o J-Link, borre **solo** el rango del firmware y cargue después:

```
> erase 0x08000000 0x0803FFFF
> loadbin firmware.bin 0x08000000
```

Si el sector 7 llega a perderse: nada se rompe. En el siguiente arranque el firmware detecta el anillo en blanco, lo formatea y arranca con valores por omisión — simplemente repite `CT`, `LC`, ajusta su algoritmo y zona horaria, y `ES`. Pierde el ajuste fino, no el aparato.

Antes de la primera grabación de una unidad terminada, haga una copia completa (ejemplo con J-Link; ST-LINK Utility tiene el botón «save» equivalente):

```
> JLinkExe -device STM32F411CE -if SWD -speed 4000
> savebin backup_full.bin 0x08000000 0x80000
> exit
```

Restaurar: `loadbin backup_full.bin 0x08000000`.

2.3 Grabación, método A — ST-Link (el más simple)

1. Conecte el ST-Link V2 a los pines SWD del BlackPill (SWDIO, SWCLK, GND, 3V3).
2. Arduino IDE → Tools → **Upload method: "STM32CubeProgrammer (SWD)"**.
3. Pulse Upload. Listo. El programador escribe solo lo que ocupa el sketch.

2.4 Grabación, método B — DFU por USB (sin programador)

> **¿Qué BOOT0 tiene su placa?** Los BlackPill de WeAct antiguos llevan un jumper BOOT0; los actuales (v3.1) solo un **botón BOOT0**. Use la receta que corresponda.

Placas con jumper: ponga el jumper BOOT0 a **1** y pulse RESET — la placa aparece como "STM32 BOOTLOADER". Tras la subida, devuelva el jumper a **0** y pulse RESET.

Placas con botón (v3.1), la forma fiable: desconecte cualquier alimentación externa; **pulse y mantenga BOOT0**, conecte el cable USB **manteniendo el botón**; a los pocos segundos suelte BOOT0 — Windows no debería avisar de «dispositivo no reconocido» y STM32CubeProgrammer verá la placa. Suba el firmware y pulse RESET. (Mantener BOOT0 mientras se pulsa NRST, como sugieren otras páginas, normalmente **no** funciona — sobre todo con la placa ya montada en un PCB.)

Windows puede pedir un driver la primera vez — véase la nota siguiente.

2.5 Tras cualquier grabación: la nota del USB

- Cuando el IDE termine, **pulse el botón RESET de la placa** — el dispositivo USB re-enumeración limpiamente del modo bootloader al CDC serie del firmware.
- El serie USB del firmware es **VID 0483, PID 5740** ("STM32 Virtual COM Port"). Si Windows se niega a abrirlo, instale el driver una vez con Zadig.

Parte 3 — Primer arranque: calibración, algoritmo, guardado

Hace falta un programa de terminal (PuTTY, Tera Term, el monitor serie del Arduino IDE o el sintonizador de la Parte 8 — el sintonizador es el más cómodo).

3.1 Conexión

1. Conecte el USB (y/o el Bluetooth a 57600 — en la configuración de fábrica ambos funcionan a la vez).
2. Abra el puerto a **115200, 8N1**. Fines de línea: CR o LF, ambos valen.
3. Pulse **Intro**. Debería ver el prompt o el inicio del informe de 1 Hz.
4. Escriba `H` y pulse Intro. Se desplaza la lista completa de comandos — es la Parte 7 de este manual, en vivo.

El registro de arranque, línea a línea. Los primeros diez segundos tras el reset imprimen una secuencia fija. Este es uno típico de una unidad sana ya configurada (las líneas exactas dependen del hardware), y después, qué significa cada una:

```
================================================
GPSDO v1.08.71rt compiled 2026-10-29 23:34:12  build 71
image CRC32 8EED4610  (303364 bytes of flash)
FreeRTOS port by J. M. Niewinski  with Claude, GLM-5.3 Max & Qwen3.8-Max AI
https://github.com/jmnlabs/GPSDO_FreeRTOS
Inspired by GPSDO v0.06c by Andre Balsa
https://github.com/AndrewBCN/STM32-GPSDO
Algos 0-2 original design by Andre Balsa
Algos 3-9 by J. M. Niewinski
Algo 10 (LTIC 3-stage) inspired by Dan Wiering's measurements
Algo 11 (LTIC-Lars) after Lars Walenius' PI loop
Algo 12 (multi-level accumulator) after Alan Cashin (MIS42N)
Algo 13 (Kalman filter) by J. M. Niewinski - original to this project
Type H = help  SW = stack diagnostics
================================================
Reset cause: POWER-ON/BROWN-OUT PIN/NRST
Flash ring: sector 7 ready
Settings: recalled from flash ring
Live store: LRN + LC applied from flash ring
Initial PWM=44653 algo=12 time_offset_min=120
GPS init: probing baud rate...
GPS detected at 38400
GPS: disabling noisy NMEA at 38400 baud
GPS: 4/4 NMEA sentences disabled
GPS: sending UBX config at 38400 baud
UBX: CFG-NAV5 ACK
LEA-T: accepted CFG-TMODE2 (28B)
Hardware configured, creating RTOS objects...
Timers started
Starting FreeRTOS scheduler
HW: AHT10/AHT20 sensor    OK  (I2C 0x38)
HW: BMP280 sensor         OK  (I2C 0x77)
HW: INA219 sensor         OK  (I2C 0x40)
HW: LM75 sensor           OK  (I2C 0x48)
HW: LTIC phase input      enabled (PA1) - needs the ramp detector hw
TFT: init start (SPI1 PA5/PA7, CS=PB13 DC=PB12 RST=PB15)
TFT: freq-band sprite (4-bit) created
TFT: header sprite (4-bit) created
TFT: data sprite (1-bit) created
```

Después arranca el informe de 1 Hz (Parte 6). Lo que dice cada línea:

Línea	Significado
rótulo (`GPSDO v1.08.71rt …`)	identidad del firmware — versión v1.08, build 71 (`.71rt`). Si ve caracteres basura: velocidad equivocada — use 115200
`image CRC32 …`	suma de comprobación del programa en la flash, calculada al arrancar: identifica el binario que realmente corre, aunque la hora de compilación de arriba esté desfasada (véase 1.8)

Línea	Significado
`Reset cause:`	por qué se reinició el chip. `POWER-ON/BROWN-OUT PIN/NRST` = encendido o botón normales. `SOFTWARE` = reset ordenado por el firmware (`RB`, fin de una subida). `INDEP-WDG/WINDOW-WDG` = watchdog (este firmware no usa ninguno — trátelo como señal de fallo). La línea extra `-> supply dipped: check the 3V3 rail under load` aparece tras un brown-out: el carril de 3V3 cayó bajo la carga del OCXO — arregle la alimentación, no lo ignore
`Flash ring: sector 7 ready`	el anillo de ajustes del sector 7 se encontró válido
`Flash ring: sector 7 blank/formatted (defaults)`	anillo virgen o borrado — **normal en la primera grabación**; el firmware lo formatea y usa los valores de compilación
`Settings: recalled from flash ring`	sus ajustes guardados (algoritmo, parámetros del lazo, zona horaria, flags) se aplicaron
`Settings: none stored (compile-time defaults)`	nada guardado aún — lo esperable en una unidad nueva; recorra la Parte 3 y ejecute `ES`
`Live store: LRN + LC applied from flash ring`	datos aprendidos (calibración LC, modelo de deriva, último punto de trabajo) aplicados — la línea solo aparece con una unidad ya calibrada
`Initial PWM=… algo=… time_offset_min=…`	punto de trabajo elegido al arrancar: código PWM final (los datos aprendidos pisan los ajustes si son más recientes), algoritmo restaurado y desfase de zona horaria en minutos
`GPS init: probing baud rate...` / `GPS detected at …`	el receptor se encuentra y se mide su velocidad (prefiere 38400)
`GPS: no response to baud probe, defaulting to 9600`	**ningún receptor respondió** — revise el cableado del módulo GPS antes que nada
`GPS: disabling noisy NMEA…` / `GPS: sending UBX config…` / `UBX: CFG-NAV5 ACK`	el receptor pasa a modo binario y dinámica estacionaria; las variantes `NAK`/`no response` son supervivibles — el receptor simplemente conserva su configuración
`LEA-T: accepted CFG-TMODE2`	un receptor de tiempo (clase LEA-M8T) quedó configurado en survey-in / Time Mode — solo aparece con `GPSDO_GPS_TIMING` y receptor de tiempo
`Hardware configured…` / `Timers started` / `Starting FreeRTOS scheduler`	arranque interno; las tres deben aparecer siempre
`HW: <sensor> OK (I2C …)` / `HW: <sensor> not found`	el escaneo del bus I2C: cada sensor informa presente o ausente. Un sensor ausente no es fatal — la unidad funciona sin él, solo ese campo queda vacío en los informes
`HW: LTIC phase input enabled (PA1)`	el firmware se COMPILÓ con la ruta del detector de fase y PA1 está configurado para ella — no puede distinguir un detector montado de un pin flotante, así que esto no es una comprobación de hardware (los algoritmos 10–13 dependen del detector real)
`TFT: init start (…)`	comienza el arranque de la pantalla. **Si tras esta línea no sigue nada**, el cableado TFT o el `User_Setup.h` está mal (Parte 1.4) — es el caso «pantalla blanca/muerta pero el serie vive»
`TFT: … sprite FAILED — direct-draw fallback`	la pantalla funciona pero redibuja más despacio (faltó RAM para los sprites antiparpadeo) — cosmético, no fatal

Líneas que aparecen **más tarde**, con el aparato funcionando, y su significado:

Línea	Significado
`LTIC: running UNCALIBRATED (run LC)`	hay un algoritmo 10/11/12 activo pero `LC` nunca se ejecutó — los números de fase son nominales, no medidos. Ejecute `LC`
`LTIC ACQ: polarity unset — run 'LPOL -1' (or +1)`	el algoritmo 10 se niega a actuar hasta que fije la polaridad (espera con seguridad en vez de adivinar). Los algoritmos 11 y 12 asumían **+1** y actuaban igualmente; ahora esperan igual
`picDIV: armed (output stopped, waiting for 1PPS sync)`	el divisor quedó armado; un hueco de 1–1,2 s en su salida es lo esperado antes de sincronizar al siguiente flanco de PPS
`picDIV: no GPS fix or no 1PPS - arming deferred until both are back`	se pidió un armado sin fix, o sin pulso en los últimos 1,5 s (build 68): la petición espera y se cumple cuando vuelven los dos
`GPS 1PPS lost — auto-holdover engaged` / `GPS fix lost — auto-holdover engaged`	la referencia se fue — 10 s sin pulso, o el fix — y el lazo se congeló solo (A.5); `GPS fix recovered — auto-holdover disengaged` llega cuando el fix vuelve con 10 s de pulsos sin interrupción. Bajo `MH` la línea termina en cambio en `manual holdover kept`, y el holdover sigue siendo suyo (build 68)
`LEA-T: … survey … %/s) — continuing anyway`	monitor del survey-in; «continuing anyway» significa que el survey agotó el tiempo por encima del objetivo de precisión — normalmente cielo tapado
`!!! FreeRTOS …` (configASSERT / STACK OVERFLOW / MALLOC FAILED)	el firmware capturó un fallo, con archivo/línea o nombre de tarea — el LED azul parpadea rápido al mismo tiempo. Anote ambas cosas; la unidad necesita reset (`RB`)

Una unidad virgen pide calibración automáticamente en el primer arranque — puede ver la cuenta atrás de calibración antes que cualquier otra cosa.

3.2 Deje que el GPS se asiente

- Dé a la antena una vista limpia del cielo. Un alféizar sirve; una antena geodésica en la azotea sirve mejor.
- Con un receptor de tiempo (M8T/F9T...) y `GPSDO_GPS_TIMING` compilado, al arrancar corre el **survey-in**: el receptor mide su propia posición durante al menos 300 s hasta que la estimación baja de 5 m, y entonces pasa a **Time Mode** en posición fija. Desde entonces la pantalla muestra `HDOP:TIME` en vez de un número. El survey-in solo debe completarse una vez; se repite tras perder alimentación. (`SV 0` lo apaga, `SV 1` lo reactiva, en el siguiente arranque.)

Cómo dejar el survey fijo. El survey de 300 s / 5 m que el firmware corre al arrancar es un compromiso: bastante largo para servir, bastante corto para que nadie lo espere. Si puede dejar el receptor encendido durante horas, haga el survey una vez en u-center y guárdelo en la configuración del módulo respaldada por batería — u-center **V8.29** tiene los comandos adecuados (UBX-CFG-TMODE2 para el survey y luego UBX-CFG-CFG → *Save current configuration*). Un survey largo da una posición fija mejor y sobrevive a los cortes de alimentación: el firmware comprueba el Time Mode al arrancar y se salta el suyo cuando el receptor ya informa de uno. Es la recomendación de Alan Cashin, y la precisión más barata de todo el montaje. - El calentamiento del OCXO tarda 300 s (`WU 0` lo salta; déjelo activo). - El LED amarillo está **apagado sin fix GPS, fijo con fix** — cuando algo se vea mal, mire ahí primero.

¿Qué módulo GNSS elegir? Un receptor de tiempo u-blox genuino (clase LEA-M8T) es la base para la que está construido este firmware: survey-in, Time Mode, la corrección de diente de sierra `qErr`. También disciplinará con módulos de navegación baratos —incluidos los clones chinos de u-blox de eBay/AliExpress— porque al lazo le basta el pulso 1PPS y las frases NMEA. Pero los clones ignoran la configuración binaria: espere `0/4 NMEA sentences disabled`, tramas CFG sin ACK y unos ~15 s más de arranque mientras expiran los timeouts; no hay survey-in ni `qErr`, la pantalla muestra un HDOP numérico para siempre, y el vagabundeo extra de la posición se cuela en la fase — justo lo que absorben los límites holgados por omisión del algo-12. Una rareza conocida de los clones: tras la ronda de configuración fallida, algunos dejan de emitir datos y la pantalla se queda en "acquiring" — la cura es entrar en el túnel de u-center (`T` con el baud del módulo, p. ej. `T 9600`) y simplemente dejar que expire: la nueva sonda tras el túnel restaura el funcionamiento. Un conmutador de firmware para Un conmutador de firmware lo cubre todo: defina **`GPSDO_FAKE_UBLOX`** en `gpsdo_config.h` y el firmware solo sondea el baudrate y no envía nada más — un solo interruptor, al margen de las demás opciones de GPS activas. Aún sin probar en hardware (el autor no tiene ningún módulo clon); se agradecen informes de campo.

3.3 Calibración — primero `CT`, después `LC`. El orden importa.

Estos dos comandos enseñan al firmware dos hechos físicos distintos, y el segundo necesita al primero:

Paso 1 — `CT` (sensibilidad del oscilador + ajuste del lazo, ~3 minutos; ~11 con una planta suave). Escriba `CT` e Intro. El firmware lleva la tensión de control por tres puntos (1,5 V; 2,0 V; 2,5 V), mide la frecuencia en cada uno, ajusta una recta y calcula **K — cuántos hercios vale un paso PWM en *su* oscilador** (rango aceptado 0,01–2 mHz/LSB). Con K deriva ganancias PID razonables para los algoritmos 3–9 y los lazos LTIC, y **guarda él solo el grupo PID** al flash. Con una planta suave — EFC de span reducido o DAC externo, K por debajo de 0,12 mHz/LSB — una segunda pasada vuelve a medir alrededor del punto de 10 MHz con promedios de 100 segundos, lo que añade unos ocho minutos. No corte la alimentación mientras dure. En una placa con puente de span (PB14), ejecútelo en la posición que use; la otra posición recibe su propio `CT` sola la primera vez que el puente vaya allí.

Paso 2 — `LC` (calibración del detector de fase, ~5 minutos). Escriba `LC` e Intro. El firmware arma el divisor picDIV, centra el detector, recorre una rampa del condensador y mide los **ns por voltio**, el **offset de cero en voltios** y el **rango en ns** del detector. Con PASS **se guarda solo**. Se niega a ser útil si `CT` no ha corrido — por eso el orden importa. Si el divisor deja la fase por debajo de la banda del detector, `LC` espera a que su propio barrido la meta en ella (build 68: dos minutos como mucho, `LC: ramp wait … s` cada 10 s, un segundo armado si no basta), así que en una placa así tarda algo más. Sólo arma con un fix GPS y un pulso 1PPS al que sincronizar el divisor: sin ellos termina con `LC: ERROR no GPS fix or no 1PPS to sync the picDIV to — aborting.` y restaura el código del lazo — revise la antena y vuelva a lanzarlo.

Comprobación: escriba `LL` y confirme que el bloque LTIC muestra `ns_per_volt`, `zero_offset`, `range_ns` distintos de cero. Escriba `DAC` — hasta le dirá cuántos microhercios vale un paso de salida.

Cómo se obtiene una lectura (build 62). Justo antes de cada flanco del PPS el firmware detiene todo lo demás en el chip — también las interrupciones, durante unos quince microsegundos alrededor del flanco — y desde 5 µs después del flanco hace 64 conversiones del condensador directamente sobre los registros del convertidor, unos 110 µs en total; ajusta una recta a ellas frente a sus tiempos en TIM2 y la lee en el propio flanco, antes de que se haya fugado nada de la carga. `LB` muestra la última ráfaga: "taken at the edge", 64 códigos que bajan, la recta, la dispersión alrededor de ella (2–3 códigos rms es lo normal), lo rápido que bajan como constante de tiempo (unos 1,4 ms — es sobre todo el propio convertidor: cada conversión se lleva cerca del 0,12 % de la carga, ~3 códigos; la fuga sola es mucho más lenta) y su curvatura junto a la típica de la placa (unos −3 códigos en la placa del autor). Toda ráfaga se curva un poco, igual cada segundo; solo se rechaza la que se aparta 3,5 códigos o más de la curvatura de las demás. Una conversión que se aparta de la recta más de seis códigos (y cuatro MAD) se deja fuera y la recta se recalcula, hasta veinte veces desde el build 70 (doce desde el build 67, antes ocho): un detector que capta una onda cuadrada recibe un par de perturbaciones en cada uno de sus flancos — en la placa de Dan cuatro o cinco pares por ráfaga, separados unos 21 µs, de algo ajeno al GPSDO que conmuta a unos 23 kHz (siguen con el pin del PWM en bajo, y van y vienen según la hora del día) — y con doce se rechazaba una ráfaga con seis pares o más. Muchas ráfagas rechazadas como "off the line" en `LB` apuntan a una perturbación así en el detector. Desde el build 71 una ráfaga en la que una de cada tres conversiones se aparta a la vez pierde primero ese tercio entero, antes de descartar ninguna conversión suelta: en algunas ráfagas las conversiones se suceden cada 16⅔ periodos de los 10 MHz (en otras cada ~17,7), así que una de cada tres los encuentra en el mismo punto de su periodo, y un detector que capta sus flancos tiene un tercio de la ráfaga desplazado los mismos pocos códigos — en la placa de Dan, 17–30 códigos arriba o 7,5 abajo. Ese tercio apartaba la recta varios códigos, o más de veinte conversiones quedaban fuera de la recta una a una y la ráfaga se rechazaba. Se deja fuera cuando se aparta de los otros dos más de 6 códigos y tres veces el ruido propio de la ráfaga, esos dos coinciden, y dos de cada tres de sus conversiones están al menos a la mitad de esa distancia; los veinte descartes quedan para las perturbaciones. `LB` dice entonces `every third conversion from #0 left out: +29.6 codes off the other two`, y si no, a qué distancia está el tercio más alejado (`thirds kept: the furthest -0.5 codes off the other two`). La curvatura típica de la placa se aprende solo de ráfagas cuya recta cae dentro de la banda medida por `LC`: un detector contra su raíl (picDIV aún sin sincronizar, la fase fuera de la ventana) da ráfagas curvadas unos +70 códigos, y en el build 64 le enseñaron +20, tras lo cual se rechazaban las ráfagas propias de la placa (corregido en el build 65). Y desde el build 71 le sigue el paso a la curvatura cuando cambia: en la placa del autor crece al bajar la lectura, unos 0,19 códigos por código, así que un salto que aleja la fase 60–100 ns durante minutos curva cada ráfaga de otro modo que la típica, y la típica la seguía a lo sumo a un treintaidosavo de código por segundo — 131 ráfagas rechazadas por curvadas en 16 minutos de los dos saltos del 27.10. Ahora, cuando la mediana de la curvatura de las últimas 15 ráfagas dentro de la banda se aparta más de 3,5 códigos de la típica, la típica pasa a ella de inmediato (en un modelo de esos minutos, 14–21 rechazadas en vez de 270–290); una curvatura que no se

mueve se sigue como antes. Justo después de la ráfaga el condensador se descarga por PA1 (`GPSDO_LTIC_ACTIVE_RESET`), así que el siguiente pulso lo carga desde 0 V sea cual sea la temperatura; la fuga sola, más lenta en una placa fría, podría dejar parte de la carga de un segundo bajo la lectura del siguiente. Antes de la ráfaga imprime contadores desde el arranque: si se captó el flanco, si la recta se sostuvo (y si no, por qué) y cuántas veces se mantuvo la lectura anterior — que se mantiene, junto con el qErr del receptor que le corresponde, hasta 3 s cuando no se captó el flanco o se rechazó la recta, y cuando la puerta de valores atípicos retiene un salto; desde el build 71 también en cuántas ráfagas se dejó fuera un tercio y cuántas veces la curvatura típica se movió de golpe (`every third conversion left out 12, typical bend moved 3`). `LRM 0` toma en su lugar la mediana de dieciséis conversiones más lentas, como hasta el build 60 (para comparar; no se guarda). Los dos modos leen la fuga en momentos distintos, así que una calibración `LC` pertenece al modo en que se hizo: tras pasar al build 62, ejecute `LC` una vez más.

Mientras se espera una ráfaga — desde 150 ms antes de cada flanco hasta 2 ms después — la línea de comandos retiene su salida: el texto que salía por Bluetooth en el flanco subía la lectura de ese segundo hasta ~20 ns. Un listado largo como `LB` sale, por tanto, con una breve pausa cada segundo; no se pierde nada.

3.4 Elegir el algoritmo

- **`LA 12`** — el acumulador multinivel (según Alan Cashin). La opción probada: sujeta la fase a unos pocos ns RMS, corrige de promedio cada pocos minutos. **Es la recomendación.**
- **`LA 11`** — el lazo PI continuo de Lars Walenius. El clásico maduro y suave; excelente comportamiento a largo plazo con una sola perilla (`LTC`).
- **`LA 13`** — el filtro de Kalman (parte 4.6). El más nuevo: decide cuánto creer a cada lectura a partir de las varianzas y no de una constante, y toma cada número que necesita de `CT` y `LC`. No hay nada que ajustar. **El simulador lo aduló** — en el banco real lo limita la deriva lenta del cero de su propio detector y en la métrica de lectura de fase queda tras el algoritmo 11 (véase 4.6a, por qué esa métrica puede ser injusta con él y qué pregunta queda abierta). Más nuevo que el 12, menos horas de hardware — la recomendación de arriba sigue en pie; el 13 es el que vigilar.
- `LA 10` — lazo de fase en tres etapas (ACQ→DPLL→LOCK). Sólido, con más parámetros.
- Los algoritmos 0–9 son la colección histórica — funcionan, están congelados, véase la Parte 4.

`LA` a secas muestra el algoritmo actual. Elegir **no guarda** — véase el paso 3.6.

3.5 Extras recomendados

- **`SAW 1`** — corrección de diente de sierra. El error de cuantización del 1PPS del receptor (±8…±25 ns, reportado cada segundo como `qErr`) se resta de la fase medida. Con un receptor de tiempo es precisión gratis; actívelo.
- **`TZ <ciudad>`** — hora local con DST (p. ej. `TZ Madrid`, `TZ Adelaide`), o `TO 1` para desfase fijo, `LT 1` para mostrar hora local en vez de UTC. `H TZ` explica el formato de regla si falta su zona.
- **`PO <Pa>`** — offset añadido a la lectura de presión del BMP280 (−5000..5000).
- **`AO <m>`** — offset añadido a la **altitud del GPS** en pantalla y telemetría (−3000..3000 m). Corrige la altitud mostrada a la realidad del terreno, p. ej. si el modelo de geoide se equivoca 30 m: `AO 30`.

3.6 Guardar todo — `ES`

Escriba `ES` e Intro. Esto escribe **todos** los ajustes más los datos aprendidos al flash ring del sector 7. Desde este momento un corte de corriente no pierde nada.

Un puñado de comandos-preferencia se guardan **ellos solos** en el momento de fijarlos y se lo dicen — la respuesta trae `[auto-saved: …]`. Son: `TZ`, `TO`, `LT` (grupo zona horaria), `WU`, `SPL`, `SV` (flags), `PO`, `AO` (offsets), además de `CT` (guarda él solo el grupo PID que acaba de derivar) y un `LC` aprobado (se guarda al anillo live).

Todo lo relativo al **lazo** — ganancias (`KP/KI/KD/IL`), todos los parámetros LTIC y Lars, los ajustes del algo-12, y la propia elección `LA` — vive en RAM hasta que lo guarde. Cada uno de esos comandos responde con una pista tipo `[not saved — run 'ES LTIC' to keep it]`: ejecute el guardado de grupo indicado, o simplemente recuerde: **tras una sesión de ajuste, `ES`**.

3.7 Cómo se ve un aparato sano

- La palabra de tendencia (pantalla + línea `PWM:` del informe) llega a **`LOCK`** y se queda — con el algo 12, destellos breves de `CORR`/`ZC` son **normales y sanos** (una corrección es el algoritmo auto-comprobándose con calendario; la pantalla aun así los cuenta como enganchado).
- La fase (`dph:` del informe) deambula en torno a cero dentro de decenas de ns y siempre vuelve — no puede irse de rampa.
- Las ventanas de frecuencia se cierran: `100s:` en milihercios, `1ks:` en microhercios, en una o dos horas.
- `CS` (estadística de correcciones) imprime números RMS pequeños y estables que no crecen hora a hora.
- En el TFT, el número grande de frecuencia está **verde**.

En un banco tranquilo con el algo 12, espere: fase 5–20 ns RMS, correcciones de unos pocos LSB cada pocos minutos, desviación de Allan en la clase 1e-11…1e-12 desde 1000 s hacia arriba. Junto a un radiador, una puerta que se abre o el sol directo los números serán peores — eso es física, no un fallo.

Parte 4 — Los catorce algoritmos (0–13) y sus parámetros

Una idea subyace a todo esto: el firmware cuenta la frecuencia del OCXO con un contador enventanado (TIM2), mide la fase GPS-contra-OCXO con el detector LTIC y ajusta la tensión de control (el «DAC» PWM, 65536 pasos, ~48,8 µV por paso a 16 bits; con `GPSDO_PWM_DITHER` la resolución efectiva son 24 bits — unos 0,2 µV por paso). Convención de signo: error medido `e = freq − 10 MHz`; si `e > 0` el oscilador va rápido y el PWM debe **bajar** (para un EFC de sensibilidad positiva; `LPOL` / la gestión de polaridad cubre los EFC invertidos).

> Si fase, error de frecuencia o PID son palabras nuevas para usted, deténgase aquí y lea el Apéndice A (qué persigue el lazo y por qué debe ser suave) y el Apéndice B (qué hacen de verdad las letras P, I y D). Diez minutos ahí vuelven legible cada parámetro de abajo.

La línea `Learn:` del informe nombra el algoritmo activo; el sintonizador elige de ahí la familia de gráficas automáticamente.

4.0 El menú

#	Nombre (como se muestra)	En una frase	Estado
0	primitive	el controlador escalonado original de André Balsa, ciclo 429 s	por defecto tras un flash frío
1	forced-drift	+1 LSB por 1000 s — caracterización del oscilador	diagnóstico
2	random-walk	ruido ±1 LSB cada 5 s — medición del suelo de ruido	diagnóstico
3	FLL-PID-man	PID sobre la media de 100 s	clásico
4	PLL-PI-man	PI sobre fase, ciclo 10 s	clásico
5	PLL-PID-man	como el 4 con su propio hueco de ajuste	clásico
6	FLL-PID-gen	FLL PID, ajuste por algoritmo genético	clásico
7	PLL-PID-gen	el viejo PLL de batalla; su Kp almacena el resultado de CT	clásico
8	hybrid-FLL-PLL	mezcla sigmoide del 6 y el 7 según el tamaño del error	clásico
9	NN-MLP	red neuronal pequeña + conducción térmica aprendida en holdover	clásico
10	LTIC-3stage	máquina de estados ACQ→DPLL→LOCK sobre el detector de fase	línea recomendada
11	LTIC-Lars	lazo PI continuo de Lars Walenius	línea recomendada
12	multi-level	acumulador multinivel de Alan Cashin	**la recomendación**
13	kalman	filtro de Kalman de tres estados — fase, frecuencia, envejecimiento	el más nuevo

Los algoritmos 0–9 están **congelados**: permanecen porque funcionan y porque el hueco de ajuste del 7 almacena además la calibración CT. El desarrollo nuevo es solo 10/11/12/13.

Selección: `LA <n>` (persistente con `ES ALGO`). Una unidad recién flasheada arranca siempre con el algoritmo 0 — elija el suyo una vez, guarde, y queda para siempre.

4.1 Algoritmos 0–2 (diagnóstico)

Sin parámetros. El 0 mueve el PWM cuando las medias de frecuencia cruzan umbrales fijos. El 1 lleva el PWM en rampa lineal (caracterizar el EFC). El 2 inyecta ruido (medir el suelo del lazo). No los necesitará en uso normal.

4.2 Algoritmos 3–9 (los clásicos) — parámetros por KP/KI/KD/IL

`KP <algo> <val>` / `KI` / `KD` fijan las ganancias (algoritmos 3–7, 0..100000); `IL <algo> <val>` la pinza del integrador (algoritmos 3–9, 100..100000). `LP [n]` lista. Valores por omisión (y la base de la que parte `CT` para derivar los suyos medidos):

algo	Kp	Ki	Kd	I_LIMIT
3	70,0	0,70	175,0	9000
4	1000	0,020	2,0	7000
5	1000	0,020	2,0	10000
6	205	0,264	14950	13000
7	1000	0,020	2,0	10000
8	—	—	—	13000
9	—	—	—	450

Extras: el algoritmo 8 tiene `BC` (cruce de mezcla, por defecto 0,024 Hz) y `BS` (escala de mezcla, 0,012 Hz) — el centro y el ancho de la sigmoide que decide cuánto FLL contra PLL mezclar a cada error; el algoritmo 9 tiene `NS` (paso máximo, por defecto 175 LSB). Guardar: `ES PID`. Sinceramente: tras `CT` no debería necesitar tocar nada — para eso está CT.

4.3 Algoritmo 10 — LTIC de tres etapas

Máquina de estados sobre el detector de fase: **ACQ** (captura guiada por frecuencia, además recentra la rampa del detector) → **DPLL** (ciclo 2 s, frecuencia + fase) → **LOCK** (correcciones cada `LIV` segundos con la estimación de par de ventanas).

Parámetros (todos `ES LTIC`; `LL` lo lista todo):

Comando	Significado	Rango / omisión
`LAT`	umbral de fase de ACQ	ns, por defecto 100
`LDT`	umbral de deriva DPLL→LOCK	1e-13..1,0, por defecto 5e-10
`LIV`	intervalo de corrección en LOCK	1..600 s, por defecto 300
`AQP/AQI/AQD/AQL`	PID de la etapa ACQ	0..100000
`DPP/DPI/DPD/DPL`	PID de la etapa DPLL	0..100000
`LKP/LKI/LKD/LKL`	PID de la etapa LOCK	0..100000
`LPOL`	polaridad PWM→fase, −1/0/+1	0 = sin fijar: el lazo espera
`LCV`	objetivo de centrado de ACQ	0 (= auto, centro del rango) .. 3,3
`ACG g [cap]`	ganancia [tope de paso] del centrado	50..20000 LSB/V [5..1000 LSB]
`FA/FAD/FAL`	ventana de media de frecuencia del término amortiguador (ambas / DPLL / LOCK)	10, 100 o 1000 s

`ltic_autotune` (parte de CT/LC) rellena las ganancias por etapa a partir de la sensibilidad medida, así que la secuencia de arranque descrita ya ajusta este algoritmo. Si ACQ nunca sale, revise `LPOL` — el lazo avisa y espera hasta que lo fije.

4.4 Algoritmo 11 — LTIC-Lars, lazo PI continuo

El lazo de Lars Walenius, portado con fidelidad. Un controlador PI evaluado cada segundo, con prefiltro adaptativo; lock cuando la fase filtrada se mantiene dentro de `LPL` ns durante `LTC × LPF` segundos. Detector al riel → captura automática por frecuencia, con un rearmado de picDIV permitido.

Comando	Significado	Rango / omisión
`LG`	ganancia del lazo. **0 = auto desde CT** (recomendado)	0..10000
`LD`	amortiguación	>0..1000, por defecto 3
`LTC`	constante de tiempo del lazo — **la única perilla**	1..600 s, por defecto 60
`LFD`	divisor del prefiltro (prefiltro = LTC/esto)	1..100, por defecto 2

Comando	Significado	Rango / omisión
`LTO`	objetivo de fase en el detector	0..3,300 V, por defecto 2,111 V
`LPL`	ventana de fase del lock	1..10000 ns, por defecto 100
`LPF`	factor de retención del lock (hold = LPF × LTC)	1..100, por defecto 5
`LTK`	coeficiente térmico, pasos DAC por paso ADC	−32000..32000, 0 = apagado
`LTR`	referencia de temperatura	0..3,300 V

El ajuste en una frase: deje `LG 0`, ponga `LTC` según lo suave que quiera el lazo (60 s es buen arranque; 240 s apacigua un sitio ruidoso; más corto solo añade ruido), y el resto por omisión. Todo con `ES LTIC`. La pestaña **LTIC-Lars** del sintonizador expone exactamente estos campos.

4.5 Algoritmo 12 — el acumulador multinivel

Puerto del GPSDO de Alan Cashin (MIS42N), refinado durante v1.04–v1.06. Sin ciclo fijo: **el tamaño del error elige el tiempo de promediación.**

Cómo funciona, en palabras llanas. Cada segundo la lectura de fase entra en una escalera de acumuladores. El nivel *n* promedia 2^(n+1) segundos — la escalera corre 2, 4, 8, … 2048 s (11 niveles). Cada nivel guarda un par de valores (A = mitad vieja, B = mitad nueva). Al cerrarse el par se calculan dos números: la **pendiente** (B − A = error de frecuencia en ese tramo) y la **fase extrapolada** (3B − A = la fase ahora mismo). Si la fase cabe en el límite del nivel, el par se suma y asciende un nivel (doble promediación). Si lo excede — corrección inmediata, la escalera se reinicia, y la corrección se reparte suavemente en el tramo del que vino (mínimo 64 s — «un GPSDO quiere que un error grande se corrija con suavidad»; el suelo original de Alan es 16 s, elegido para que una corrección en caliente no exija una tensión de control fuera del rango del DAC — un suelo menor da pasos mayores, uno mayor da una recuperación de fase más lenta). Independientemente de eso, alcanzar el nivel `MR` (por defecto 7 = 256 s) fuerza una corrección aunque todo esté bajo los límites — si no, una deriva lenta jamás recibiría respuesta.

Cada corrección lleva hasta tres partes, y es a propósito: cancelar el error de **frecuencia** medido, devolver la **fase** a cero, y nunca una sin la otra. Después, el truco que Alan llama esencial: el empuje deliberado queda apuntado, y a la primera lectura de fase que cruza cero en la dirección esperada se retira exactamente ese empuje (**ZC**, cancelación en el cruce por cero) — el oscilador queda con la frecuencia correcta *y* sin error de fase.

La prueba de cruce por cero se arma **solo tras una corrección disparada por un límite**, que es la regla de Alan. Una corrección programada (el nivel `MR`) salta por reloj, con la fase donde esté, así que no hay sobreoscilación que esperar; y un ZC no se rearma a sí mismo, porque el ZC *es* la cancelación. Armarla en cualquiera de esos dos casos dejaría que un cruce ajeno, minutos después, sacara un paso de una pendiente ya caduca.

Tendencias propias del 12: `WAIT` (aún sin datos) → `SYNC` (5 s de asentamiento tras armar el divisor) → `FLL` (detector al riel, captura por frecuencia) → `NOPH` (fase no válida, PWM congelado) → `ACQ` → `LOCK` (>16 s de quietud genuina **y** frecuencia en puerta; las correcciones y los ZC *no* reinician el contador de quietud — son salud, no ruido). Los destellos transitorios `CORR` / `ZC` son normales. `NoCT` significa: ganancia en auto pero `CT` nunca corrió — ejecute `CT`. `NoPL` significa: polaridad del EFC sin fijar — el lazo espera hasta que ponga `LPOL -1` o `1`.

Parámetros (`ES ALGO12`; `ML` lista todo, incluida la estimación de ruido 1-sigma en vivo):

Comando	Significado	Rango / omisión
`MG`	ganancia, LSB por ns de fase. **0 = auto desde CT**	0..10000, por defecto 0 (auto)
`MR`	nivel que fuerza una corrección	0..10, por defecto 7 (256 s)
`MM`	cómo llega una corrección al pin: 1 = tranquilo (abajo), 0 = la ruta original	0..1, por defecto 1
`MF`	origen de los límites por nivel: 0=seguir MG, 1=tabla guardada, 2=fórmula sigma, 3=ajuste medido	0..3, por defecto 0
`MFT`	con MF 3: segundos objetivo entre correcciones por ruido	0 (=3600) o 2048..65535 s

Co-mando	Significado	Rango / omisión
`MLP n v`	una fila de la tabla de límites de 11 niveles	nivel 0..10, valor en nanosegundos

Modo tranquilo (`MM 1`, por defecto desde el build 60). Con la fase ya dentro de `LAT` cambian dos cosas y la aritmética de la jerarquía no: del nivel de 128 s hacia arriba el término de frecuencia se aplica entero en vez de a la mitad; y la salida avanza hacia una corrección a no más de 1 LSB por segundo (un cambio de más de 32 LSB — adquisición, FLL, rearme — sigue siendo inmediato), con el cruce por cero retirando exactamente el slew que aplicó, fracción incluida. El build 60 hacía además que los niveles bajo esa frontera necesitaran el triple de su límite y ataba la frontera a `MR`. Con ambas cosas la placa registró 6,1 ns con `MR 7` y 9,5 con `MR 10`, frente a 2,7 del algoritmo 11 esa misma semana; en una simulación calibrada con esas ejecuciones, sin ellas salen 5,2 ns con `MR 7` y 5,1 con `MR 10` (`MM 0`: 6,1 y 5,6), así que el build 61 quitó ambas. El PWM se mueve cinco veces menos que con el algoritmo 11, que sigue sujetando la fase unas dos veces más; los pulsos son la forma en que este algoritmo mueve la fase. Por debajo de `MR 6` ninguna corrección llega a 128 s y el término de frecuencia sigue a la mitad. `MM 0` es la ruta original, sin cambios, para un A/B en una misma placa.

La tabla de límites por omisión es la del propio Alan (reescalada de su detector de 25 ns a este de ~1 ns). En nanosegundos por tramo: L0 462, L1 400, L2 331, L3 264, L4 191, L5 164, L6 126, L7 117, L8 108, L9 103, L10 63 ns. `ML` la marca **UNTUNED** (solo la fila de 128 s derivó alguna vez de una especificación) — trátela como el punto de partida probado, no como escritura. Su origen, en palabras del propio Alan, es un **escenario de recepción en el peor caso**: los números se calcularon para un NEO-6 con antena dentro de una habitación, cuyo 1PPS puede desviarse ±100 ns en ratos cortos (±30–40 ns de promedio). La tabla es holgada a propósito — dimensionada para el peor receptor posible, no para el hardware clase LEA-M8T que esta construcción suele llevar.

¿Qué MF elegir? Resultados de campo, dos placas:

- **Sitio tranquilo (antena de alféizar en casa, temperatura estable):** la fórmula sigma (`MF 2`) o la medida (`MF 3`) funcionan — el lazo se autoescala y corrige cada pocos minutos.
- **Sitio ruidoso (taller con puerta, cerca de un radiador, aire en movimiento):** el autoescalado puede cazar en el nivel 0 (límites más estrechos que las deambulaciones reales del entorno). Allí, la tabla holgada de Alan (`MF 1`) fue dramáticamente más estable. Si ve correcciones de nivel 0 constantes, cambie a `MF 1`.

Detrás de ambos resultados está el principio de diseño declarado por Alan, conveniente conocer antes de ajustar: *«no queremos que el algoritmo trabaje con límites, queremos que trabaje con correcciones programadas a intervalos definidos»*. La corrección por nivel de ejecución (`MR`) debe ser el motor, y los límites solo una red de seguridad para el calentamiento y las excursiones de recepción — y exactamente por eso su tabla es holgada. Una tabla estrecha que dispara sin parar trabaja, según esa filosofía, contra el algoritmo y no con él.

La ganancia (`MG`) pertenece al **oscilador**; los límites pertenecen al **ruido del sitio** — por eso los fijan comandos distintos, y para eso está `MF`: para poder expresar «ganancia medida, límites a mano».

En el sintonizador: la pestaña **Multi-level (algo 12)** tiene los cuatro escalares y la tabla completa de límites de 11 filas con botones Send/Read/List, y las gráficas en vivo cambian solas a la familia del algo-12 (`ph` error de fase, nivel, contador de correcciones, sigma, contador de ZC).

4.6 Algoritmo 13 — el filtro de Kalman (original de este proyecto)

Los algoritmos 10, 11 y 12 responden a la misma pregunta — *¿cuánto de la lectura de fase de este segundo debo creer?* — con un número decidido de antemano: la etapa de una máquina de estados, una constante de tiempo, un nivel en una tabla de umbrales. Éste la responde a partir de las varianzas, y la vuelve a responder cada segundo.

Lleva tres estados — **fase**, **frecuencia**, **envejecimiento** — cada uno con su covarianza. Sabe cuánto ruido tiene su detector porque lo mide, y con qué rapidez deriva su oscilador porque también lo mide, así que el peso que da a cada lectura es el que esos dos números digan en ese momento. A tiempos de promediado cortos, donde el detector es ruidoso y el OCXO está tranquilo, se apoya en el oscilador; a tiempos largos, donde el OCXO camina, cede ante el GPS.

Lo que cuesta. Tres estados y una medida **escalar**, así que la inversión de matrices de los libros es una sola división: unas 140 multiplicaciones-acumulación y 36 bytes, una vez por segundo — aproximadamente un microsegundo del M4F. La

afirmación de que Kalman exige un Cortex-A o una FPGA se refiere a filtros GNSS de veinte estados, no a éste.

El holdover sale gratis. El estado lleva frecuencia y envejecimiento con sus covarianzas, así que perder la fase no es un caso especial: deja de actualizar, sigue prediciendo y sigue gobernando. La tendencia muestra `NOPH`.

Usa dos medidas. La fase del detector LTIC y la frecuencia del contador enventanado TIM2. Esta segunda no es un refinamiento: con el picDIV sin sincronizar no hay ninguna fase válida, y un filtro con sólo medida de fase se queda entonces **sin** medida — predice desde un estado todavía en cero y el oscilador se va. TIM2 es además lo que hace verificable al detector: fase y frecuencia son la misma magnitud derivada, así que en una ventana la fase **tiene** que moverse lo que suma el error de frecuencia. Un detector que no se mueve cuando TIM2 dice que debe no está midiendo nada, y el lazo rearma el picDIV y gobierna sólo con la frecuencia hasta que se comporte.

Todo se deriva de `CT` y `LC`. Nada en él es una constante medida en el banco de otro: el cuanto propio del detector (`ns_per_volt × 3,3/4096`) fija la semilla del ruido de medida y su suelo, media banda del detector fija la covarianza de arranque en frío, la banda cruzada en un horizonte fija la de frecuencia, y `R/KT³` siembra el ruido de proceso. Vuelva a ejecutar `CT` o `LC` y los números siguen en un segundo. Al arrancar, el filtro imprime una línea con lo que concluyó:

```
KAL: from CT/LC  res 1.01ns  R0 2.52ns  Q0 0.000006600  P0 1500ns  lim 939LSB  arm<0.25Hz
```

Dos ruidos de proceso, no uno. El modelo de reloj que implementa este filtro lleva `Sf`, el ruido blanco de frecuencia que aparece como paseo aleatorio en fase, y `Sg`, el paseo aleatorio de frecuencia: `Q = [[Sf·t + Sg·t³/3, Sg·t²/2], [Sg·t²/2, Sg·t]]`. Hasta el 01.09 aquí solo existía `Sg`, y esa única omisión era toda la sobreactuación: sin `Sf`, la única forma de explicar una fase que se movió más de lo previsto es decidir que deriva la *frecuencia*, lo que sube la ganancia que escribe el estado de frecuencia, que es lo que el DAC sigue. `Sf` ahora se mide en vez de adaptarse — las diferencias de fase a retardo 1 y a retardo 16 llevan `sigma_R² + Sf·k/2`, así que dos retardos separan ruido blanco de paseo donde uno no puede — y una diferencia que no supera dos sigmas de su propio ruido de estimación se reporta como cero, que con un detector limpio es la respuesta correcta. `KL` lo muestra junto a `R` y `Q`.

Q se adapta, pero no puede correr más que su horizonte. El filtro ajusta el ruido de proceso `Q` hacia lo que hace que sus propias innovaciones salgan del tamaño que predijo. Eso tiene un sesgo sistemático, y en el banco mordió: cuando el error del detector está *correlacionado* — un cero que deriva en minutos — las innovaciones son mayores de lo que el filtro espera por una razón que nada tiene que ver con el oscilador, así que `Q` sube segundo tras segundo hasta que la covarianza crece lo bastante para explicarlas. Se detiene, pero se detiene alto. El 30.08 la placa corría con `Q` 195x su semilla: un filtro cinco veces más rápido que el horizonte sobre el que se le mandó gobernar, moviendo el DAC 1,80 LSB por segundo frente a los 0,49 del algoritmo 11 esa misma noche — y sin mejor fase a cambio.

`(R/Q)^(1/3)` es un tiempo, y es la constante de tiempo del propio filtro; `Q = R/KT³` es exactamente decir *corre tan rápido como el horizonte que te dieron*. Ahí queda ahora acotada la adaptación: **el filtro no puede correr más rápido que `KT`**, con la `R` medida y no la sembrada — que es lo que hace que la regla diga lo mismo en cualquier placa. Medido sobre dos plantas, cinco semillas de ruido y tres niveles de deriva del detector: ADEV a tau corto el doble de bueno, movimiento del DAC 2,4× menor, a cambio de un 7% en la sd de fase. Nada se pierde al negar que `Q` absorba el error correlacionado del detector, porque `R` ya lo tiene: `R` se mide de diferencias tomadas con dieciséis segundos de retardo, así que un cero que deriva en esas escalas está en el ruido de medida, donde le corresponde. ¿Quiere un lazo más rápido? Acorte `KT`: ahora es lo único que lo mueve. Un valor que fije usted con `KQ` pasa tal cual: la cota es sobre la adaptación, no sobre usted. `KL` escribe `[at ceiling]` cuando está apoyada en el techo. Eso solo no dice que `KT` sea demasiado largo, aunque hasta el build 70 lo decían este manual y `KL`: lo que empuja Q hacia arriba viene tanto de la deriva lenta del lado GPS, o del detector, como del oscilador — y un lazo más rápido la copia —, y en la placa de Dan Wiering (28–29.10) `KT 1000` se quedó en el techo y aun así fue mejor que `KT 500` — ADEV plana en $2{,}9\cdot10^{-12}$ de 40 a 400 s, frente a una joroba de $4\cdot10^{-12}$ a 100–150 s. Compare valores de `KT` por la ADEV, un turno con cada uno.

Parámetros — los cuatro son opcionales y los valores por defecto son los que se usan:

Comando	Por defecto	Significado
`KR [ns]`	`0` = medir	Ruido de medida. Cero significa "mídalo de las diferencias del propio detector a un retardo de dieciséis segundos" — ruido blanco y deriva lenta del cero juntas, que es lo que quiere. Fíjelo sólo para inmovilizar el filtro en un experimento.
`KQ [v]`	`0` = adaptar	Ruido de proceso, $(ns/s)^2$ por segundo. Cero significa "adáptalo de la secuencia de innovaciones".
`KT [s]`	`100`	Horizonte de fase: con qué rapidez el control anula la fase estimada. Más corto sigue más al GPS, más largo se apoya en el oscilador. Fija también la paciencia de la prueba de bloqueo: cinco horizontes lejos y se rearma el picDIV. Desde el build 67 un arranque en frío usa como mucho 100 s de él (véase `KT` más abajo).
`KL`	—	Lista el estado: fase, frecuencia y envejecimiento, cuánto cree cada uno, la R y la Q en uso, y cuántas lecturas descartó la compuerta de innovación. `[at ceiling]` junto a Q significa que la adaptación está contra su cota.

`KR`, `KQ` y `KT` se guardan en el momento de teclearlos, en su propio registro del flash ring — sin `ES`, y un firmware más antiguo sencillamente nunca pide ese registro.

La línea Learn dice

```
Learn: algo=13 (kalman) ph=-12.4ns f=-118.30ps/s sig=2.48ns R=2.64 rej=3 arm=1
```

— la fase y la frecuencia que el filtro **cree** (no la lectura de este segundo, que es justamente el sentido de tener un filtro), cuánto cree la fase, el ruido de medida que ha medido, cuántas lecturas rechazó la compuerta de innovación y cuántas veces ha rearmado el divisor. `HOLD=<s>` aparece cuando funciona sólo con el modelo, y `KF` mientras una perturbación se cierra con el horizonte `KF` (build 69).

En el sintonizador: la gráfica superior muestra la estimación de fase del filtro con `Vphase` y sus guías de banda debajo — porque la pregunta que este lazo plantea más a menudo es si el detector está vivo.

Medido en el simulador (`tools/loopsim`, cinco semillas de ruido, dos plantas, ruido blanco del detector): sd de fase **0,81 / 1,20 ns**, frente a 3,07 / 4,20 del algoritmo 12 y 3,62 / 7,39 del algoritmo 11. **El banco no está de acuerdo, y el banco tiene razón**: repetir esas mismas plantas con deriva lenta del cero invierte el orden (11: 7,45 → 10,07 ns; 13: 1,22 → 7,23), y en hardware real del 29–30.08 la misma placa dio 2,96 ns (11) frente a 5,2–5,9 ns (13) con un suelo plano en los tiempos medios de promediado — el nivel de la deriva del propio detector, independiente del ancho de banda del lazo. El ruido del simulador era demasiado blanco; trate sus cifras como relativas, nunca absolutas (4.6a).

4.6a Cómo transcurre un segundo del lazo — y qué mueven las perillas

Un segundo, en orden. *Predicción*: el estado (fase, frecuencia, envejecimiento) avanza un segundo. Después hasta dos *mediciones* lo corrigen — la fase del detector LTIC (salvo en raíl, congelada o fuera de banda; y salvo durante los cuatro segundos de silencio deliberado tras un arm, cuando la rampa lee el raíl y eso no es una fase) y la frecuencia del TIM2 (siempre; mantiene vivo el lazo cuando el detector está ciego). Finalmente el *control*: `u = -(freq + fase/KC)` — cancelar el error de frecuencia estimado y anular la fase estimada sobre el horizonte **del controlador** `KC` (vigente desde el primer bloqueo del lazo; antes, en adquisición, al paso tranquilo de `KT`), con pinza a la banda del detector; el resto sub-LSB pasa al segundo siguiente. La corrección solo puede crecer poco a poco (su tamaño máximo permitido repartido en un horizonte), para que un reinicio no sacuda el oscilador, pero puede reducirse tan rápido como pida el filtro — antes del build 65 no podía, y un enganche desde medio hercio de distancia llevó el PWM de un raíl al otro (véase el registro de cambios). Lo que realmente llegó al pin se contabiliza de vuelta en el estado de frecuencia. Alrededor del núcleo: la puerta de innovación 4σ, la prueba de confianza (¿se mueve la fase como TIM2 exige? — ignorando titubeos menores que la propia resolución del contador), el arm de referencia de arranque y el holdover.

La visión del R honesto. La R que informa `KL` (~2,9 ns en este banco) se mide de las propias diferencias del detector a un retardo de dieciséis segundos: el suelo *blanco* (~2,5 ns) más el crecimiento de la deriva lenta del cero durante esos 16 s. La deriva **completa** del cero es una estructura aparte (~2,6 ns en ~45 s en este banco) que el filtro **deliberadamente no persigue** — por eso el algoritmo 13 se niega a seguir la estructura lenta del detector que el 11 sigue sin preguntar. Medido: con el lazo quieto, dph muestra un suelo plano de 5–9 ns de 10 s a 600 s de promediado **con independencia**

del ancho de banda del lazo — ancho, adaptado y rígido aterrizaron en el mismo nivel, el nivel de la deriva del propio detector. Si seguir esa deriva (11) o negarse (13) da mejor *salida* sólo lo puede responder un patrón independiente.

`KR [ns]` (por defecto 0 = medir). Fijarlo al suelo blanco (`KR 2.5`) hace que el filtro siga al detector por completo: el sd@600 mejoró ~30% en horas tranquilas, pero la puerta 4σ se estrecha con R y los eventos GPS reales (decenas de ns) se rechazan — los rechazos subieron de 163 a 5021 por noche. Fíjelo sólo para acotar un experimento; `KR 0` es el mejor omiso.

`KQ [v]` (por defecto 0 = adaptar). La adaptación funcionaba como un trinquete: el error correlacionado del detector mantiene las innovaciones por encima de lo que la covarianza predice, así que **Q subía** — 195× la semilla tras una noche, un filtro cinco veces más rápido que su horizonte moviendo el DAC 1,80 LSB/s frente a los 0,49 del algoritmo 11. Ahora está acotada en **ocho `R/KT³` —** el filtro no puede correr más rápido que `KT` **— lo que reduce 2,4× el movimiento del DAC y duplica el ADEV a tau corto (4.6). La placa de Dan Wiering, mismo firmware y sin tocar nada, llegó hasta el viejo raíl: `Q` 1000× su semilla en 2h37m, el DAC moviéndose 11,05 LSB/s y un ADEV a 20 s de 8,6e-11 frente a 3,1e-12 del algoritmo 11 en la misma placa y el mismo rubidio. `KL` imprime `[at ceiling]` cuando la adaptación está contra la cota. Fijar en la semilla (`KQ 0.000006359` — la semilla exacta está en la línea `KAL: from CT/LC`) rigidiza el lazo aún más y casi congela el PWM; un valor que fije usted pasa tal cual, la cota es sobre la adaptación.** KQ se guarda al te-clearlo** — devuélvalo con `KQ 0`.

`KT [s]` (por defecto 100; 200 en el build 60). Desde el build 36 es el horizonte **del estimador** — la paciencia del entendimiento, no la velocidad de las manos. Un KT más corto es un ancho de banda de creencia mayor (y una cota de Q más alta, porque el límite es `R/KT³`); uno más largo se apoya en la estabilidad propia del oscilador. Es también la paciencia del detector de estancamiento — cinco horizontes lejos de casa y el picDIV se rearma. El ritmo de anulación de fase es ahora trabajo de `KC`. Qué lado de 100 s es el correcto depende del oscilador: el del autor deriva rápido y quiere 100 s; un OCXO tranquilo, medido contra un rubidio, puede querer 300–1000 s, y la nota del comando `KT` por encima de 200 s dice cómo elegir: por la ADEV, un turno con cada valor — no por Q en su techo en `KL` (build 71; hasta el build 70 la nota decía que entonces se acortara `KT`, y en la placa de Dan `KT 1000` se quedó en el techo y superó a `KT 500`). Desde el build 67 la captura usa `KT` sólo hasta 100 s — el horizonte del control, la primera Q, la hipótesis sobre la frecuencia, la puerta de armado —, así que un `KT` largo ya no le cuesta nada a un arranque en frío: en el modelo de una placa tranquila `KT 1000` enganchaba antes en 28–41 minutos y ahora en 6–12, y `KT 100` exactamente como antes.

`KC [s]` (por defecto 0 = auto = KT/3, 33 s con el `KT` por defecto — KT/6 en los builds 61–67, KT en el build 60, KT/3 antes; **vigente sólo desde el primer bloqueo del lazo** — en adquisición y tras reiniciar el algoritmo el control vuelve a KT, como mucho 100 s desde el build 67). Con qué rapidez las *manos* anulan un error de fase que el *entendimiento* ya conoce: la estimación ya está suavizada, así que anularla rápido amplifica la corrección, no el ruido. Medido en un log de 20 horas: ADEV de salida ~23% mejor en tau 256–4096, con el dither del DAC subiendo de 0,37 a 0,52 LSB/s — un presupuesto, no un almuerzo gratis. **Júzguelo por el oscilador, no por la fase registrada.** La fase registrada es la lectura del detector, y unas manos rápidas la mejoran copiando en el oscilador el propio vagabundeo lento del detector — y lo que sale de la placa es el oscilador. Lo que hace el oscilador es el PWM integrado: sus pasos por los hercios por LSB, cuya ADEV coincidió en unos pocos por ciento, de 4 a 400 s, con la medida de la placa de Dan frente a un rubidio (14.10). En la placa del autor, en una sola sesión, `KC 16` una noche y luego `KC 33` dos noches y un día: el PWM cambió 664–744 veces por hora en lugar de 1349, y esa ADEV fue 1,7 veces menor a 4–16 s, una quinta parte menor a 64 s e igual a 256 s, a cambio de una quinta parte más de fase registrada (desviación típica 2,14–2,30 ns frente a 1,83) — el valor por defecto desde el build 68. KT/6 (builds 61–67) se había elegido sólo por la fase registrada: una noche con `KC 16` frente al algoritmo 11 la noche anterior, 2,63 frente a 2,73 ns, con el PWM cambiando unas 2,5 veces más a menudo, en pasos de 1–4 LSB. `KT 200` / `KC` = `KT` del build 60 registró 5,62 ns: se eligió en una simulación sin el ruido rápido del propio oscilador, y un controlador tan lento deja pasar ese ruido. En la simulación calibrada contra la placa, `KC 25` da más o menos la fase del algoritmo 11 con 1,4 veces su movimiento del DAC, y por debajo de ~15 s la fase gana poco mientras el DAC se mueve mucho más. (El barrido anterior que dio por inestable un `KC` largo corría sobre un simulador que leía mal la fracción del DAC — ver el changelog del build 60.) Una placa conserva lo guardado, y teclear cualquiera de `KR`, `KQ`, `KT`, `KC` o `KF` guarda los cinco — una placa en la que se tecleó alguno con el build 60 arranca con `KT 200`; `KT 100` + `KC 0` restauran el valor por defecto. `KC` sin argumento muestra el valor configurado y el realmente vigente.

`KF [s]` (build 69; por defecto 16 s, 10–250 s, `KF 0` = desactivado). El horizonte que usa el controlador mientras dura una **perturbación**. `KC` es una elección sobre el ruido; un salto de la frecuencia del oscilador — una caída de la alimentación,

un golpe, un salto del cristal — no es ruido, y un controlador lento le deja empujar la fase otro tanto. El 21.10 una caída de tres minutos del raíl de 5 V de la placa del autor (a 4,37 V) movió el OCXO $2{,}7 \cdot 10^{-9}$: con `KC 33` la fase llegó a +75 ns, bajó a −51 ns al volver la tensión y no volvió dentro de ±10 ns hasta 12 minutos después; reproducido con `KF 16`, +41/−29 ns y 5½ minutos. El modo se activa cuando la estimación de fase del filtro supera 10 veces el ruido en calma de la placa (20 ns en la del autor, unos 37 ns en un detector tan ruidoso como el de Dan) o cuando la compuerta cede tras diez lecturas rechazadas, y se desactiva tras dos minutos seguidos por debajo de 3 veces ese ruido. Actúa sobre la estimación, no sobre la lectura, nunca hace el controlador más lento que `KC` y, tras un arranque o un armado del picDIV propio, espera a que la fase se asiente (un `AP` tecleado en la consola cuenta como perturbación). El log dice cuándo cambia el horizonte (`KAL: disturbance (|phase| 43 ns) - controller 33 -> 16 s`, `KAL: calm for 120 s - controller 16 -> 33 s`), la línea `Learn:` termina en `KF` mientras está activo, y `KL` lo muestra. El funcionamiento en calma no lo ve. Medido en la placa del autor (26–27.10, saltos `SP` de +82 y +90 LSB): con `KF 16` la fase llegó a −58 y −62 ns y estaba dentro de ±10 ns a los 4,4 y 4,0 minutos; con `KF 0`, a −86 y −96 ns, dentro de ±10 ns por primera vez a los 13 y 16 minutos, luego +14 y +12 ns hacia el otro lado, dentro para siempre a los 25–27 y 32 minutos. Cada vez la compuerta cedió 18–26 s después del salto.

`KL` — lea el estado antes, durante y después de cada experimento: estimaciones, sigma de la creencia en la fase, la R y la Q realmente en uso (`[at ceiling]` junto a Q significa que la adaptación ha llegado a su cota de `R/KT³` — en un sitio con microeventos GPS incesantes eso es la mayor parte de la noche, propiedad del cielo y no avería), la última innovación, el contador de rechazos, el **ratio de adaptación** (innovaciones mayores que lo predicho, >1 empuja Q arriba, <1 abajo — rotulado `Q pushed up`, `Q pushed down` o `consistent` desde el build 71; hasta el build 70 `wants more Q` y `wants less`, que sonaban a consejo sobre `KT` y no lo son) y las **marcas de agua de Q** desde que empezó el tracking (`lo..hi` — si Q salió alguna vez de la cota durante la noche). El hábito más informativo: `KL`, luego `SW`, luego un log de una hora.

Tendencias: `KAL` (normal), `REJ` (la puerta rechazó una lectura — en singles o parejas en eventos GPS es lo normal), `KEPT` (el detector repitió su última lectura y el filtro no tomó fase de ella — alrededor de un segundo de cada mil en lock, uno de cada tres durante un enganche; build 66), `ARM` (re-arm del divisor — arranque, raíl o estancamiento), `NOPH` (sin fase, gobierno desde el modelo; **cuatro segundos de NOPH justo tras un `ARM` son silencio deliberado** — la rampa lee entonces el raíl, y eso no es una fase), `NoPL`/`NoCT` (falta calibración), `WAIT` (aún sin datos). Cómo se ve una buena noche — véase el Apéndice D.

Parte 5 — Pantallas: qué significa cada campo

Pueden funcionar varias pantallas a la vez (I2C + SPI no chocan). Todas muestran la misma verdad con distinto nivel de detalle.

5.1 La TFT (480x320 o 320x240)

```
y=  0..23   barra de título: "GPSDO v1.08"   CPU 58%   LMT 14:32:45 Thu

El `CPU 58%` de la barra de cabecera es la carga total del procesor,
siempre activa (la medición corre en la tarea de uptime; `TL` en la
línea serie añade el desglose por tarea). Aparece desde el segundo
segundo tras el arranque — la ventana de promediado de 100 s aún no
tiene veredicto.
y= 30..62   FREQUENCY — dígitos grandes, con color
y= 70..151  rejilla de datos, dos columnas
y=156..195  fila de sensores
y=204..239  barra de estado
```

El color del número grande de frecuencia es el control de salud de un vistazo:

Color	Significado
verde	enganchado (para 10/11: tendencia `LOCK`; para 12 también durante `CORR`/`ZC` — un lazo que corrige es un lazo sano)
blanco	ajustando
naranja	holdover
rojo	sin señal / sin fix

Durante los procedimientos de arranque, el número se sustituye por cuentas atrás naranjas: `Survey <s> ±<m>`, `OCXO warmup <s>`, `Tune <s>` (CT), `LTIC cal <s>` (LC), `Calibrate <s>`.

Rejilla de datos: columna izquierda — hora UTC + día de la semana, fecha, uptime (contado desde el 1PPS del GPS, no deriva), `Algo: n <tendencia>`, código `PWM:` + tensión `Vct:`. Columna derecha — `Sat:` + `HDOP:` (o `HDOP: TIME` tras el survey-in — esa palabra es *buena*, significa modo de tiempo en posición fija), `Lat:`/`Lon:` (6 decimales), `Alt:` + `qErr:` (el valor de diente de sierra que se resta ahora mismo), `INA:` tensión y corriente de alimentación. Filas de sensores: `BMP:` temperatura + presión, `AHT:` temperatura + humedad, `Vph:`/`dph:` tensión del detector y fase en ns (`ovf` = rampa fuera de su banda válida), `Vcc:`/`Vdd:` carriles de alimentación.

Barra de estado (fondo de color). El verde significa una sola cosa: el lazo de control ha convergido, según el mismo veredicto que colorea las cifras de frecuencia encima — la barra y las cifras no pueden contradecirse.

barra	color	qué significa
`DISCIPLINED FIX OK`	verde	fix bueno, lazo enganchado
`ACQUIRING FIX OK`	naranja	fix bueno, el lazo trabaja pero aún no ha llegado
`OCXO WARMUP`	naranja	el oscilador todavía no merece ser juzgado
`CALIBRATING`	naranja	`C`, `CT` o `LC` está moviendo la salida a propósito
`HOLDOVER (manual)`	naranja	congelado con `MH`
`HOLDOVER (fix lost)`	rojo	el fix desapareció, o el 1PPS durante 10 s (build 68), y el lazo se congeló solo
`WAITING FOR GPS FIX`	rojo	no hay contra qué disciplinar

Se añade `SURVEY` mientras corre un survey-in. El verde tarda en llegar y se va deprisa a propósito: el veredicto del lazo es cosa de un segundo y puede parpadear por una sola cuenta del contador, así que la barra quiere cinco segundos consecutivos de enganche antes de ponerse verde y dos de pérdida antes de soltarlo.

5.2 La OLED (128x64) — dos páginas que alternan cada 10 s

Página A (GPS): hora local, frecuencia, Lat/Lon/Alt+Sats, uptime, UTC+temperatura AHT, PWM + tendencia (una `H`/`A` parpadeante al borde derecho = holdover manual/auto). Página B (sensores): filas BMP/AHT/INA, Sat+HDOP, UTC. Durante los procedimientos, la fila de frecuencia muestra `F SVIN <s>s <m>m`, `F WARMUP <s>s`, `F CAL <s>s`.

5.3 El LCD 20x4

Línea 0 frecuencia; línea 1 UTC + días de uptime; línea 2 rota cada 10 s (coordenadas / sats+HDOP / AHT / INA / BMP); línea 3 PWM + Vctl + tendencia con el marcador parpadeante de holdover.

5.4 Los dígitos de reloj (TM1637 / HT16K33)

Hora local HH:MM (los dos puntos parpadean al segundo). Todo rayas = arranque/sin datos; `oooo` = sin fix; molinillos = calentamiento / survey / calibración.

Desde el build 67 los molinillos casi no cuestan nada: la tarea de pantalla se despierta unas siete veces por segundo para moverlos y, en esos despertares, sólo toca los dígitos LED; la TFT, la OLED y la LCD se redibujan una vez por PPS, como siempre. Antes cada despertar lo redibujaba todo y la cifra de `CPU` se duplicaba durante el calentamiento y el survey-in (55–56 % frente a 27 % en la placa del autor).

El brillo se fija al compilar, uno por tipo de pantalla, en `gpsdo_config.h`: `TM1637_BRIGHTNESS` (0-7) y `HT16K33_BRIGHTNESS` (0-15). Ninguno tiene comando CLI - se ajustan una vez, para el módulo instalado y la luz del sitio.

5.5 Los LED de la placa

LED	Significado
Amarillo (PB8)	APAGADO = sin fix GPS *y* sin holdover · fijo = fix OK · parpadeo lento (1 s) = holdover manual, con o sin fix · parpadeo rápido (200 ms) = fix o 1PPS perdido, holdover automático
Azul (PC13)	**solo fallos** — parpadeo rápido significa que el firmware capturó una avería (aserción / desbordamiento de pila / memoria agotada), con el motivo por el serie. *No* es un latido; un LED azul apagado es el estado bueno.

Parte 6 — Telemetría serie (el informe de 6 líneas)

Una vez por segundo (una por PPS) el firmware imprime un bloque de estado con esta forma (modo legible, `RH`):

```
Up: 000d 02:15:33  UTC: 22/8/2026 14:32:45
Lat: 51.477928 Lon: -0.001531 Alt: 46.5m Sat:10 HDOP:TIME
Freq: 10000000.0000 Hz  10s:0.0  100s:0.02  1ks:0.000  10ks:0.0000
PWM:44653  Vctl:1.970V hit
Learn: algo=11 (LTIC-Lars) gain=auto scale=46 phase=12.3ns LOCK qErr=-8.2ns
BMP:23.4C 1013.2hPa  AHT:22.1C 45.3%rH  INA:12.05V 250mA  Vphase:3.077V dph:1390.5ns  CPU:31%
```

(La posición del ejemplo es el Real Observatorio de Greenwich — longitud cero por definición, un sitio público a propósito. Su unidad imprimirá por supuesto sus propias coordenadas; si comparte registros, recuerde la opción **Redact position** del sintonizador.)

Línea a línea:

Línea	Campos
1	uptime (contado desde el PPS, fiable desde la v1.06) + fecha/hora UTC del GPS
2	posición (6 decimales), altitud (altitud GPS **más su `A0`**), satélites, HDOP — o la palabra `TIME` cuando el receptor de tiempo está en modo de posición fija; sin fix → `GPS: no position fix yet`
3	frecuencia contada en bruto (o `---` antes del primer conteo) y las ventanas promediadas de error 10 s / 100 s / 1 ks / 10 ks — cada una aparece solo cuando su búfer se ha llenado
4	código PWM de 16 bits, tensión de control medida, palabra de tendencia (`hit`, `ACQ`, `DPLL`, `LOCK`, `PLL`, `CORR`, `ZC`, `NOPH`, `NoCT`, `ARM`, `NoPL`, …) — en holdover, `[HOLDOVER]` sustituye a la tendencia; `RAILS` la sustituye mientras el lazo va de un extremo del DAC al otro sin enganche entre medias (build 65 — véase la Parte 10)
5	línea Learn según el algoritmo (abajo) + `qErr` cuando `SAW 1` está activo
6	temperatura/presión BMP280 (**bruta + su `P0`**), temperatura/humedad AHT, tensión/corriente INA, tensión del detector `Vphase`, fase `dph` en ns (con el diente de sierra restado, igual que el propio lazo) y `CPU:` — la parte del último segundo que el procesador no pasó ocioso. Si al arrancar se encontró un LM75 (build 68), le sigue `LM75:` — la temperatura en °C con tres decimales, `LM75:---` cuando el sensor deja de contestar. Con `TL 1` sigue un campo `TL:` que desglosa la carga por tarea.

La línea Learn por familia:

- algo 11: `gain=auto|<val> scale=<n> phase=<ns> LOCK|acq`
- algo 12: `ph=<ns> level=<n> corr=<n> arm=<n> sig=<ns> zc=<n> secs=<n>` — fase acumulada, último nivel que actuó, correcciones, armados, estimación de ruido en vivo, cruces por cero, segundos
- algo 13: `ph=<ns> f=<ps/s> sig=<ns> R=<ns> rej=<n> arm=<n>` y `HOLD=<s>` en holdover, `KF` durante una perturbación (build 69) — las estimaciones del filtro, no la lectura de este segundo
- algo 10: `state=ACQ|DPLL|LOCK`
- algos 3–9: deriva/pendiente/amortiguación aprendidas; el 9 añade el coeficiente térmico aprendido

Modo con tabuladores (`RD`) imprime una línea por segundo con los mismos datos como columnas (uptime, ventanas de frecuencia, sats, HDOP, PWM, tensiones, todos los sensores, el TIC en bruto) — el formato para volcar a una hoja de cálculo. Una columna de media de frecuencia se **deja vacía hasta que su ventana se ha llenado** — los primeros 10, 100, 1000, 10 000 y 20 000 segundos respectivamente — para que una gráfica muestre dato ausente y no un oscilador marcando cero hercios. El número de campos no cambia, así que los separadores siguen ahí y un analizador que cuente columnas no se ve afectado. `RP`/`RR` pausa/reanuda. El informe no bloquea: un host que conecta y no leerá pierde colas de informes, pero no congela las pantallas (arreglado en v1.06 — una cola CDC llena llegó a clavar de por vida la tarea de pantalla).

Parte 7 — Referencia de comandos (todos)

Serie, 115200, sin distinguir mayúsculas, terminado con Intro. Los comandos con argumento `[val]` **muestran el valor actual si se invocan sin argumento**. Hay dos regímenes de guardado — el firmware siempre dice en su respuesta cuál acaba de tocar:

- **Las preferencias se guardan solas** e imprimen `[auto-saved: …]`: `TZ`, `TO`, `LT`, `WU`, `SPL`, `SV`, `PO`, `AO` — más `CT` (guarda él solo el grupo PID que deriva) y `LC` (se guarda solo con PASS).
- **Los parámetros del lazo viven en RAM hasta que los guarde** — todo lo que edita el sintonizador: ganancias, parámetros LTIC/Lars/algo-12, `LA`, `SP`. La respuesta nombra el grupo exacto para conservar el cambio (`[not saved — run 'ES …']`).

Versión, ayuda, estado

Comando	Qué hace
`V`	versión, autores, créditos — y el **CRC-32 de la imagen de flash**, calculado al arrancar desde la propia flash. A diferencia de la marca de compilación no puede quedar obsoleto: el compilador de Arduino reutiliza objetos, así que un sketch que no ha editado conserva una fecha anterior. Cuando un registro y un recuerdo no coinciden, fíese del CRC.
`H` / `?`	lista de comandos · `H TZ` = detalles de zonas horarias
`SW`	marcas de agua de pila, heap libre, uptime + su origen, ppm del MCU contra el GPS — y **carga de CPU por tarea**, de mayor a menor, promediada sobre una ventana real de 100 s de cubos de un segundo. Medida con el contador de ciclos del Cortex-M4 en cada cambio de contexto, así que es exacta y no muestreada. El tiempo de interrupción recae sobre la tarea interrumpida: léalo como "el procesador estuvo aquí", no "esta tarea consumió esto".
`TL 0|1`	las mismas cifras por tarea en la línea de telemetría, como campo `TL:`. Apagado en cada arranque y nunca guardado — es un diagnóstico de banco, no un ajuste.
(nada)	`RP` / `RR` pausan y reanudan la telemetría — un conmutador de una tecla TAB/ESC se probó y se eliminó: los terminales reales y el sintonizador envían líneas terminadas en CR/LF y nunca un TAB o ESC suelto, así que era inalcanzable desde las herramientas que la gente usa de verdad.

Salida (el DAC)

Comando	Rango	Qué hace
`SP [n]`	1..65535 (sin argumento = 32767, punto medio ≈1,65 V)	fija el DAC de control directamente — mando manual para experimentos
`up1`/`up10`/`dp1`/`dp10`	—	empujones de PWM ±1/±10 (rechazados durante una calibración)
`DAC`	`PWM`/`DITH`/`EXT`	sin argumento: informe — ruta de salida, códigos de 24 y 16 bits, Vctl **ordenado y medido**, un paso en µHz (requiere CT), y en una placa con puente de span a qué posición pertenece la planta. Con argumento: selecciona la ruta de salida activa — **se guarda solo**. El puente de la placa conmuta la señal; esto le dice al firmware qué ruta gobierna. Sin fijar = `DITH`.
`DV [v]`	2,50..5,50 V	voltios a código completo, para el "ordenado" del informe — **se guarda solo**. `3.30` (por defecto) es el modelo PWM; en una placa con AD5680 fije `5.00`. Se conservan tres decimales (`4.096` para una referencia de 4,096 V).
`AV [x]`	1,00..10,00	ratio del divisor antes del pin ADC, para el "medido" y toda visualización de Vctl — **se guarda solo**. `1.00` (por defecto) = directo.

Co-mando	Rango	Qué hace
`VS [VCC|VREF]`	—	a qué está puenteado el divisor de PA0 — **se guarda solo**. `VCC` (por defecto) es el raíl de 5 V, como en toda placa anterior a la V3. `VREF` es la referencia de tensión, y el informe `DAC` compara entonces la lectura con `DV` y protesta pasado el 5 %.
`SPAN [CLR FULL|REDUCED]`	—	el puente de span del EFC en PB14 (`GPSDO_SPAN_SENSE`): sin argumento = la posición en que está el puente y la calibración `CT` (K y nulo) que lleva cada posición. `CLR` olvida la calibración de una posición — la que se midió en el sitio equivocado. El informe `DAC` imprime las mismas líneas.

Tres rutas, un nodo. Las tres rutas de salida se pueden compilar juntas; el comando `DAC` elige cuál escribe el firmware, y el puente de la placa decide qué señal llega al filtro EFC (no hay multiplexor por software — dos controladores sobre un nodo serían un fallo de hardware). Al arrancar, toda ruta compilada se inicializa para que el conmutar no haga glitch, pero un paso de control escribe **solo la ruta seleccionada**: un AD5680 no seleccionado mantiene su último código, y mientras `EXT` está activa el pin del PWM se mantiene en bajo (desde el build 69; antes seguía reproduciendo su último código, una portadora en el filtro del PWM). `PWM` es el PWM de 16 bits simple en PB9 (en un build con el motor de dither: ese motor a granularidad de LSB enteros — el mismo voltaje, sin reconfigurar). `DITH` es la portadora de 13 bits + dither sigma-delta de 1 bit en PB9/TIM4 CH4, 24 bits efectivos en promedio temporal (portadora ≈11,7 kHz con el reloj de temporizadores de 96 MHz de la BlackPill — `DAC` imprime el de la placa —, DMA). `EXT` es el AD5680 por SPI bit-bang en PB4/PB0/PB2, escrito una vez por segundo (~20 µs con interrupciones enmascaradas).

Internamente el valor de control es **un código de 24 bits**; el valor de 16 bits del display, del anillo flash y de los logs es ese número redondeado — una vista, no una segunda variable. Las correcciones finas se acumulan en el byte bajo (un código de 24 bits que no es múltiplo de 256 es la prueba de que está mandando la ruta fina; el informe `DAC` lo marca). En `EXT` el comando se proyecta sobre la parte de **18 bits** por escalado, no por truncado: `code18 = code24 × 262143 / 16777215`. La trama SPI del AD5680 es de 24 bits — `[0000][D17..D0][00]` — con 18 significativos, así que "tensión de control de 24 bits" en la documentación significa el comando que porta el lazo (y que `DITH` realiza de verdad como promedio temporal); la parte en sí resuelve 18 bits. El escalado hace que la escala completa de 16 bits caiga exactamente en la escala completa de la parte — por eso K y todos los coeficientes medidos en una ruta pasan a la otra sin cambios. A 5,0 V un LSB del DAC son 19,1 µV; en `EXT` la referencia *es* la tensión de control, así que su deriva a baja frecuencia se proyecta 1:1 sobre la frecuencia.

La comparación «ordenado contra medido» del informe solo es honesta cuando ambas escalas corresponden a la placa: `DV` dice qué significa el código completo en la salida (3,30 V = el modelo PWM, por defecto), `AV` el ratio del divisor antes del pin ADC (1,00 = directo). En un DAC externo de 5 V con divisor en el punto de medida, fijar ambos convierte el check de supuesto-de- placa-PWM en una afirmación sobre ESTA placa — sin ellos el informe marcará MISMATCH en cada lectura. Ambos se guardan solos.

Dos spans, dos calibraciones (`GPSDO_SPAN_SENSE`, PB14). Una placa con el desplazador de nivel del EFC puede gobernar el oscilador con el span de control completo o con uno reducido, y son dos plantas distintas — en el prototipo V3 la K en la posición reducida es unas 3,5 veces menor (3,3–3,6 veces, las dos posiciones medidas sobre una misma referencia). Cablee el segundo polo del puente de span a **PB14** (puente puesto = PB14 a masa = REDUCED; quitado o sin cablear = FULL) y el firmware guardará una calibración `CT` por posición:

- cuando el puente se mueve (con antirrebote, cerca de medio segundo) los coeficientes del lazo se vuelven a derivar de la K de esa posición y el código de control se reasigna para que el pin EFC conserve su tensión — la frecuencia antes del cambio es la frecuencia después. La reasignación necesita un par de códigos que se sabe que dan la misma tensión en ambas posiciones; el firmware lo aprende solo: el código en que el lazo estaba enganchado antes de un cambio, emparejado con el código en que engancha después o con el código de 10 MHz que encuentra `CT` allí. Solo cuenta un código que de verdad es un nulo: no el que se mantiene en holdover, no uno puesto a mano (`SP`, `up`/`dp`) hasta que el lazo haya perdido el enganche y lo haya recuperado, y no uno lejos de donde `CT` encontró los 10 MHz en esa posición — tiene que estar a no más de 5e-8 del nulo de esa posición y no más cerca del de la otra (a no más de 5e-9 de su propio nulo cuenta siempre). Esta última prueba atrapa un lazo enganchado sobre la planta equivocada, que es lo que produce un banco cableado a mano cuando el desplazador entra o sale antes de que PB14 lo siga. Cada rechazo se imprime. Mientras no hay par, un cambio conserva el código y el lazo vuelve a enganchar;

- una posición que nunca se ha calibrado funciona con los coeficientes de la otra — como siempre hizo una placa de una sola calibración — mientras `CT` arranca por sí mismo en cuanto hay fix de GPS (nunca en holdover), con hasta tres intentos más 10, 20 y 40 minutos después de un fallo;
- un puente movido con la placa apagada se atiende al encender, antes de que arranque el lazo. Desde el build 65 el código guardado se remapea desde el span en el que, según la ranura del live store, se escribió, y no desde el span que nombra el registro de ajustes — son dos escrituras separadas, y un corte de alimentación entre ellas tras un movimiento, o un registro cuyo guardado falló, podían arrancar el oscilador a más de 1e-7.

`SPAN` muestra el estado, y el informe `DAC` lo imprime bajo la línea de la planta. Una placa calibrada antes de cablear PB14 tiene su única calibración asignada a la posición que leyó el pin en el primer arranque con detección de span — FULL, si el cable aún no estaba. Si esa calibración se hizo en realidad en REDUCED, el primer `CT` en la otra posición mide la misma planta, lo dice y olvida la equivocada; `SPAN CLR` hace lo mismo a mano. Las ganancias manuales en LSB (`LG`, `MG`, `LTK`) no se reescalan — un cambio de posición avisa de cada una que esté fijada.

`SPAN` imprime además el **nulo** de cada posición — el código en que `CT` encontró los 10 MHz, renovado por los pares medidos desde entonces —, que es contra lo que compara la prueba de arriba. Los nulos vienen solo de `CT`, así que una placa calibrada antes del build 58 no tiene ninguno: ejecute `CT` una vez en cada posición y la prueba queda activa. `SPAN CLR` olvida la K y el nulo de una posición, y el par.

Calibración

Comando	Duración	Qué hace
`C`	~2 min	el centrado PWM de dos puntos, más antiguo
`CT`	~3 min (+8 con K suave)	**sensibilidad K del oscilador + autoajuste de PID 3–9 y LTIC; guarda el PID él solo** — ejecútelo primero. Con una planta suave (Hz/LSB pequeños — EFC de span reducido, DAC externo) una segunda pasada vuelve a medir alrededor del código de 10 MHz ajustado: 0,8 Hz de excursión sea cual sea el span, cada punto como media de 100 s, así que K sale con alrededor de un 1 %
`LC`	~5 min	**calibración del detector de fase (ns/V, offset, rango); se guarda solo con PASS** — ejecútelo segundo
`ACG g [cap]`	—	accionamiento del centrado de ACQ: ganancia 50..20000 LSB/V, paso máx. 5..1000 LSB
`AP`	—	armar el divisor picDIV a mano

Modo e informes

Comando	Qué hace
`RH` / `RD`	informe legible / con tabuladores
`RP` / `RR`	pausar / reanudar el informe de 1 Hz
`TL 0|1`	carga de CPU por tarea en la línea de telemetría (`SW` la muestra una vez; la barra del TFT muestra el CPU% total siempre)
`MH` / `MD`	holdover (congelar el PWM, volar solo) / disciplinado
`F`	purgar los búferes de anillo del promediado de frecuencia
`T [baud]`	túnel GPS transparente por USB para u-center, 300 s (baud 4800..921600)

Selección de algoritmo y PID clásico (véase la Parte 4)

Comando	Rango	Qué hace
`LA [n]`	0..13	elegir / mostrar el algoritmo del lazo

Comando	Rango	Qué hace
`LP [n]`	algo 0..9	listar parámetros PID
`KP/KI/KD n val`	algo 3..7, 0..100000	fijar ganancias
`IL n val`	algo 3..9, 100..100000	pinza del integrador
`BC` / `BS`	0,0001..1,0 Hz	cruce / escala de mezcla del algo 8
`NS`	1..10000 LSB	paso máximo del algo 9

LTIC (algo 10) — guardar con `ES LTIC`, listar con `LL`

Comando	Rango / omi-sión	Significado
`LNV`	0..1e6	pendiente del detector, ns por voltio (la mide LC)
`LZO`	0..3,3 V	offset de cero del detector, en voltios
`LRN`	2..1e9 ns, o `0`/`1`/`R`	rango del detector, ns (lo mide `LC`; decimales y exponente valen — `LRN 2500.7`, `LRN 1e3`). El mismo comando controla el autoaprendizaje — el feed-forward de deriva y la amortiguación de los algoritmos 3–10: exactamente `LRN 0` lo congela (lo aprendido se sigue aplicando), `LRN 1` lo reanuda (activo por defecto), `LRN R` devuelve la deriva y la amortiguación a sus valores teóricos y lo guarda en el acto, `LRN` sin argumento muestra el estado. Cualquier otra cosa se rechaza. `LRN 0`/`LRN 1` se guardan con `ES FLAGS`
`LAT`	0,001.. 1e9 ns (100)	umbral de fase de ACQ
`LDT`	1e-13..1,0 (5e-10)	umbral de deriva DPLL→LOCK
`LIV`	1..600 s (300)	intervalo de corrección en LOCK
`AQP/AQI/AQD/AQL`, `DPP/DPI/DPD/DPL`, `LKP/LKI/LKD/LKL`	0..100 000	PID por etapa
`LPOL`	−1 / 0 / +1	polaridad PWM→fase (0 = sin fijar: el lazo espera y lo dice; `LC` la mide)
`LCV`	0..3,3 V	objetivo de centrado de ACQ
`LRM`	0 / 1 (1), no se guarda	cómo se lee la ráfaga del detector: 1 = una recta a través de 64 conversiones, leída en el propio flanco; 0 = la mediana de 16, como hasta el build 60
`LB`	—	contadores desde el arranque (flanco captado, recta rechazada y por qué, lectura mantenida, un tercio dejado fuera y la curvatura típica movida — build 71), luego la última ráfaga: tomada en el flanco o tarde, por qué se rechazó su recta, tiempo y código de cada conversión, la recta, su dispersión, su curvatura y la típica de la placa, a qué distancia están sus tercios entre sí, lo rápido que baja

Comando	Rango / omisión	Significado
FA/FAD/FAL	10 / 100 / 1000 s	ventana de media del término amortiguador (ambas / DPLL / LOCK)

LTIC-Lars (algo 11) — guardar con `ES LTIC`

`LG` (0..10000, 0=auto), `LD` (por defecto 3), `LTC` (1..600 s, por defecto 60), `LFD` (1..100, por defecto 2), `LTO` (0..3,300 V, por defecto 2,111 V), `LPL` (1..10000 ns, por defecto 100), `LPF` (1..100, por defecto 5), `LTK` (±32000, 0=apagado), `LTR` (0..3,300 V) — significados en la Parte 4.4.

Algo 12 (acumulador multinivel) — guardar con `ES ALGO12`, listar con `ML`

`MG` (0..10000 LSB/ns, 0=auto desde CT), `MR` (0..10, por defecto 7), `MM` (0..1, por defecto 1 = tranquilo), `MF` (0..3 origen de límites, por defecto 0), `MFT` (0=3600 s, o 2048..65535 s), `MLP <nivel> <ns>` (nivel 0..10) — significados en la Parte 4.5.

Algo 13 (Kalman) — guardado al teclearlo, sin `ES`

`KR` (0..1000 ns, 0 = medir), `KQ` (0..1, 0 = adaptar), `KT` (10..10000 s, por defecto 100), `KC` (10..10000 s, 0 = auto = KT/3; vigente tras el primer bloqueo), `KF` (10..250 s, 0 = desactivado, por defecto 16: el horizonte mientras dura una perturbación), `KL` (listar el estado del filtro) — significados en la parte 4.6.

GPS, hora, sensores

Comando	Rango	Qué hace
`SV 0|1`	—	survey-in / Time Mode apagado/encendido (aplica al próximo arranque) — **se guarda solo**
`TZ <ciudad|regla>`	p. ej. `TZ Adelaide`	zona horaria con DST (`H TZ` para detalles) — **se guarda sola**
`TO <h[:mm]|A>`	−14..+14	desfase UTC fijo, o `A` = automático desde la posición GPS (regla DST de la UE) — **se guarda solo**
`LT 0|1`	—	mostrar UTC / hora local — **se guarda solo**
`PO <f>`	−5000..5000 Pa	offset de presión añadido a la lectura del BMP280 — **se guarda solo**
`AO <f>`	−3000..3000 m	offset de altitud añadido a la **altitud del GPS** — **se guarda solo**
`SAW 0|1`	—	corrección de diente de sierra (qErr) apagada/encendida — `SAW` a secas muestra el estado; recomendado ON con receptor de tiempo (guardar: `ES FLAGS`)
`WU 0|1`	—	calentamiento del OCXO al arrancar — **se guarda solo**
`SPL 0|1`	—	animación de arranque apagada/encendida — **se guarda sola**
`BL [%]`	30..100	brillo de la retroiluminación del TFT — **se guarda solo**. `BL` a secas lo muestra.

La retroiluminación, y por qué el 100 % es el ajuste silencioso. `BL` gobierna un MOSFET de canal P en **PB5** (TIM3 CH2, 20 kHz) entre el raíl de 3,3 V y el ánodo de los LED del panel: puerta a través de ~47 Ω, 100 kΩ a masa y 10–47 µF de capacidad local en el drenador. La etapa **invierte** — puerta baja es brillo máximo — así que el firmware escribe el complemento, y `BL 100` cae en una comparación igual a cero: el pin queda estáticamente bajo y la etapa no conmuta en absoluto. Importa porque es el raíl de 3,3 V el que alimenta VDDA, y con él la referencia del ADC, la lectura de fase y el monitor de Vctl. El brillo máximo es el ajuste más silencioso, no el más ruidoso; atenuar cuesta algo de ruido en el raíl y devuelve carga y calor en una caja acoplada térmicamente al OCXO.

El suelo del 30 % existe por una razón que no es eléctrica: por debajo de un tercio aproximadamente el panel deja de ser legible en vez de atenuarse de forma útil, así que un `3` tecleado por error dejaría al operador mirando algo que igual podría ser una placa muerta. En una placa sin el MOSFET el pin simplemente lleva una onda cuadrada y no ocurre nada. `BL` se compila con cualquier TFT y **cede PB5 al generador de prueba de 2 kHz** si este se ha habilitado explícitamente: un ajuste por defecto no debe quitarle un pin a una decisión deliberada.

Guardar, recordar, reiniciar

Comando	Qué hace
`ES [obj]`	**guardar** todo (sin argumento) o un grupo: `TZ`, `PID`, `LTIC`, `FLAGS`, `ALG012`, `ALG0`, `PO`
`ER`	**recordar** — releer los ajustes del flash ahora (deshacer cambios sin guardar)
`EE`	**borrar** el hueco de ajustes — omisiones en el próximo arranque
`EW`	desgaste del flash ring: ciclos de borrado, huecos usados, sector/dirección
`FR`	estado del anillo de solo lectura (siempre activo desde v0.96)
`CS`	estadística de correcciones: recuento, pico, RMS de las últimas 100/1k/10k/100k correcciones — pequeño y estable es bueno, creciente es malo
`RB`	reinicio en caliente (conserva ajustes — pero **no** los guarda antes él solo)
`CR YES`	reinicio en frío: borra ajustes + datos aprendidos, omisiones de fábrica (el YES es obligatorio, porque el modelo aprendido tarda días en reconstruirse)

Peculiaridades conocidas de v1.08 (lista honesta)

Ninguna conocida en el momento de escribir esto. Las dos listadas para v1.07 — `LRN` leyendo su argumento como número entero (`LRN 1e3` encendía el aprendizaje, `LRN ON` lo apagaba) y `LRN R` sugiriendo un `ES FLAGS` que no guarda el reinicio — están corregidas en el build 58.

Parte 8 — El sintonizador en el PC

`tools/gpsdo_tuner.py` — una consola de escritorio para observar y ajustar. Hace todo lo que hace un terminal, más gráficas en vivo y registro CSV.

8.1 Instalación y arranque

Python 3.9 o más nuevo, luego:

```
pip install PySide6 pyqtgraph pyserial tzdata
```

(`tzdata` solo lo usa el botón *Generate tz_table.h*; en Linux/macOS el sistema ya lo trae.)

Ejecutar: `python gpsdo_tuner.py` (en Windows también con doble clic).

8.2 Conexión

Elija el puerto en la barra, baud 115200, Connect. El sintonizador lee de inmediato la versión del firmware y **todos** los parámetros actuales (`LL`, `FA`, `LP 3`–`LP 9`, todos los verbos de Lars y del algo-12, `ML`) y rellena cada pestaña — ve el estado real del aparato, no las omisiones. La línea de estado muestra `state: LOCK (locked)` etc.; los títulos de las gráficas siguen al algoritmo activo.

8.3 Las pestañas

Pestaña	Qué edita
LTIC (algo 10)	los tres cuartetos de PID por etapa; Read (`LL`) / Save (`ES LTIC`) / Revert (`ER`)
LTIC-Lars (algo 11)	`LG LD LTC LFD LTO LPL LPF LTK LTR` con botones Set
Multi-level (algo 12)	`MG MR MF MFT` + la tabla completa de límites de 11 filas (`Send limits`, `Read all`, `List (ML)`, `Save (ES ALGO12)`)
FA damping	ventanas de amortiguación DPLL / LOCK
PID algo 3-9	Kp/Ki/Kd/IL por algoritmo
Calibration	`LNV LZO LRN LCV LAT LIV LPOL`
Raw monitor	todo lo que dice la placa, sin analizar; controles de registro
Help	la referencia de comandos del firmware

Recuerde que la regla del firmware también aplica aquí: **los botones Set solo cambian RAM; el botón Save de la pestaña envía el `ES ...` que persiste.**

8.4 Las gráficas

Tres paneles, refrescados sin parar desde la telemetría de 1 Hz. Los paneles superiores dependen de la familia del algoritmo — fase (`dph`/`ph`) + tensión del detector para 10/11/12 (con el ancla gris y los bordes rojos de la banda válida, cuando la calibración se conoce), deriva + tensión de control para los clásicos. El panel inferior es siempre el error de frecuencia en Hz. Barra: selector de intervalo (1 min … all), Follow live, Clear. La resolución es el ritmo de la telemetría — lo más rápido que ~2 s es invisible, y las gráficas confían en la placa.

8.5 Registro (las gráficas no son un registrador)

Raw monitor → **Start logging**, formato Full log / CSV only / Both:

- **Full log** `gpsdo_YYYY-MM-DD_HH-MM-SS.log` — cada línea tal cual, ~217 MB por semana a 1 Hz.
- **CSV** — columnas analizadas: `utc, up_s, algo, state, dph_ns, qerr_ns, vphase_v, pwm, f10, f100, ph_ns, level, corr, sig_ns, zc, bmp_c, sat, hdop` (~65 MB/semana). Celda vacía = el campo no estaba ese segundo (nunca cero).

- **Redact position** (ON por defecto) limpia Lat/Lon/Alt del registro completo guardado — comparta registros sin publicar su tejado.

Parte 9 — Ajustes, el flash ring y el desgaste

Todo lo persistente vive en un **anillo con nivelación de desgaste** en el sector 7 del flash: 255 huecos de 512 bytes. Cada guardado (un `ES`, un `LC` aprobado, una actualización de datos aprendidos) escribe el siguiente hueco en blanco; el sector se borra solo cuando el anillo da la vuelta — una vez cada 255 guardados — y al dar la vuelta los registros más recientes de ajustes, de Kalman y live se trasladan más allá del borrado (desde el build 65; antes solo quedaba la ranura que se escribía y el siguiente arranque podía salir con los valores por defecto). Al ritmo de este firmware (incluso un banco ocupado hace ~73 guardados al día) el borrado cae cada ~3,5 días, y el flash del F411 soporta ~10 000 ciclos por sector: **unos 96 años**. No lo desgastará. `EW` muestra los contadores reales cuando dé curiosidad.

Dos clases de registro comparten el anillo:

- **Ajustes** (`ES` y compañía) — todo lo que usted eligió, guardado solo cuando usted lo pide. Los guardados parciales (`ES PID` etc.) escriben *todo* el bloque de ajustes sembrado desde el último almacenado — así guardar un grupo no puede pisar otro.
- **Datos live** — lo que el aparato aprende solo: la calibración `LC`, el modelo de deriva/amortiguación, el último punto de trabajo. Se guardan automáticamente, pero con histéresis (solo cuando algo cambió de verdad en una magnitud sensata, y como muy pronto cada 20 minutos); `LC`, `LRN R` y un cambio del puente de span los guardan en el acto. El punto de trabajo se guarda por sí mismo (desde el build 65) cuando se ha alejado más de 0,02 Hz del guardado (en LSB con la K de `CT`; 64 LSB sin `CT`) y el lazo lleva 20 minutos enganchado sin interrupción y sin saltos (un código tecleado con `SP`, `up` o `dp`, aunque sea de un LSB, o dejado por una calibración vuelve a empezar los 20 minutos) — nunca en holdover, durante una calibración ni mientras un cambio del puente de span sigue en curso; los algoritmos 0–9 no tienen enganche y nunca lo guardan así. Hasta el build 64 solo se escribía junto con el modelo de deriva, que los algoritmos 11–13 no mueven, así que un arranque en frío podía empezar con medio hercio de desvío. La ranura dice también a qué span pertenece su código; una ranura escrita por el build 64 sigue cargándose. Un guardado que la flash rechaza se reintenta a los 20 minutos.

Actualizaciones de firmware: los ajustes de v1.04/v1.05 se migran al recordar (v4→v5 añade el bloque algo-12 con omisiones); un bloque de una versión radicalmente distinta, o un hueco corrupto, se trata como «no está» — omisiones, y el firmware pide calibración en el próximo arranque. El propio anillo se auto-repara: un hueco escrito a medias por un corte de corriente falla el CRC y simplemente se salta; gana el buen anterior.

La lista práctica, una última vez:

- regrabado rutinario: seguro, los ajustes sobreviven (Parte 2),
- tras una sesión de ajuste: `ES`,
- antes de vender/regalar una unidad: `CR YES`,
- cuando la unidad se porte raro tras una sesión de experimentos: `ER` (recuperar los ajustes guardados y conocidos-buenos), y solo después diagnosticar.

Parte 10 — Resolución de problemas

Síntoma	Causa probable → solución
No compila, error `ltoa`	núcleo STM32 3.0.0 — baje a la 2.12.0 (Parte 1.1)
Compila, imprime `?` o basura donde van números	Tools → C Runtime Library → **Newlib Nano + Float printf/scanf**
TFT blanca tras compilar con el núcleo 3.0.0	lo mismo — núcleo 2.12.0
TFT blanca con núcleo 2.x	driver equivocado en `User_Setup.h` o falta `TFT_MISO PA6` (Parte 1.4)
El arranque se cuelga justo tras `TFT: init start`	igual que arriba — revise cableado y el `#define` del driver
Falta el serie USB tras grabar	pulse RESET; si sigue — driver Zadig para VID 0483/PID 5740 (Parte 2.5)
Ajustes/calibración desaparecidos tras grabar	alguien usó **Erase Chip** — es la regla del sector 7 (Parte 2.2); repita `CT`, `LC`, `ES`
LED amarillo apagado	sin fix GPS y sin holdover — antena, cielo, cable. En holdover el LED siempre parpadea, así que un LED apagado nunca significa salida congelada
`HDOP:TIME` nunca aparece	survey-in sin terminar (requiere receptor de tiempo + `SV 1` + buen cielo); se repite tras cada corte de alimentación
`CT` falla / rechaza K	cableado EFC o sensibilidad del oscilador fuera de 0,01–2 mHz/LSB — revise el búfer del DAC y el rango EFC
`LC` falla	ejecute `CT` primero; no mueva la unidad durante los 5 minutos
Tendencia clavada en `NoCT` (algo 12)	ganancia en auto pero CT nunca corrió → `CT`
Tendencia clavada en `ACQ` / `NoPL`, algoritmos 10–13	`LPOL` sin fijar (el lazo imprime el aviso y espera) — ponga `LPOL -1` o `1`, luego `ES LTIC`
El algo 12 corrige sin parar en el nivel 0	sitio ruidoso — `MF 1` (la tabla de Alan), véase la Parte 4.5
La fase se va de rampa tras `SAW 1`	no hay receptor de tiempo / qErr no válido — `SAW 0`
Las líneas del informe se congelan cuando un programa abre el CDC y no lee	arreglado en v1.06 (escrituras no bloqueantes + cola de 1 KB); si lo ve, está en firmware antiguo
LED azul parpadeando rápido	fallo de firmware capturado (aserción / pila / heap) — lea el mensaje del serie; repórtelo
La unidad se reinicia sola	revise el carril 3V3 bajo la carga del OCXO; el rótulo de arranque imprime la causa del reset
Pantalla viva, sin salida serie	¿está solo en Bluetooth? 57600 en Serial2; o alguien escribió `RP` — `RR` reanuda

Síntoma	Causa probable → solución
`LOOP: rail to rail with no lock` … en la consola, tendencia `RAILS`	el lazo llevó el PWM de un extremo del DAC al otro sin engancharse entre medias: diverge, no engancha (el algoritmo 13 del build 64 lo hacía desde medio hercio de desvío; corregido en el 65). `LA 11` engancha desde lejos; si es el propio algoritmo 11, `CT` (K) o `LPOL` están mal. Sin el detector (sin `GPSDO_LTIC`) la línea termina en «Check CT.». El aviso se borra al enganchar, al cambiar de algoritmo, cuando algo que no es el lazo pone el código (`SP`, `CT`, `LC`) o tras 30 minutos sin tocar ningún extremo

Si nada de eso ayuda: capture un registro completo con el sintonizador (Parte 8.5), anote la versión de `V` y pregunte — con el registro adjunto. Una semana de CSV a 1 Hz es la diferencia entre adivinar y saber.

10.1 Cómo informar de un problema

Casi todas las preguntas sobre este firmware se han resuelto con cuatro cosas, y sin ellas se resuelven a base de días de conjeturas en vez de minutos de lectura. Envíe las cuatro:

1. **El registro de arranque** — todo desde el reset hasta `GPS init done`, copiado como texto, no fotografiado. Dice qué placa es, qué velocidad de GPS se detectó, qué sensores respondieron, qué tramas UBX fueron confirmadas y cuál fue la causa del reset.
2. **Su `gpsdo_config.h` compilado** — o al menos la lista de interruptores que cambió. La mayoría de las sorpresas son diferencias de configuración, no fallos: una placa sin detector de fase, un segundo display en los mismos pines, `SAW 1` en un receptor de navegación.
3. **Qué módulo GNSS** — un u-blox de tiempo auténtico, uno de navegación auténtico, o un clon (Parte 3.2). Los clones se comportan de otro modo y saber cuál tiene elimina la mitad de las posibilidades de entrada.
4. **La antena y cuánto cielo ve.** «En casa, en el alféizar» es una respuesta perfectamente válida y a menudo la explicación entera.

Una comprobación antes de enviarlo: el banner debe contener una línea `compiled <fecha> <hora>`. Si no está, tiene un build anterior a v1.05 y lo primero que hay que probar es uno actual — varios informes resultaron ser fallos ya corregidos. Esa línea la imprime el propio firmware, así que no puede quedar desfasada respecto al binario como sí ocurre con un número de versión recordado.

Si la placa **se congela**, añada una observación que no cuesta nada: ¿sigue parpadeando el LED azul de PC13? Lo conmuta la interrupción de temporizador de 2 Hz y no depende de ninguna tarea, así que si parpadea el MCU está vivo y lo que se ha bloqueado es una tarea — un fallo completamente distinto al de una placa que no está funcionando en absoluto.

Apéndice A — Cómo funciona un GPSDO, en palabras llanas

Puede usar este aparato sin leer este apéndice — pero el día en que algo se porte raro, estos veinte párrafos le dirán *por qué*.

A.1 El problema

Quiere una señal de 10 MHz que esté bien **ahora** (segundo a segundo) y bien **para siempre** (a lo largo de meses y años). Dos piezas aportan media mitad cada una, y ninguna las dos:

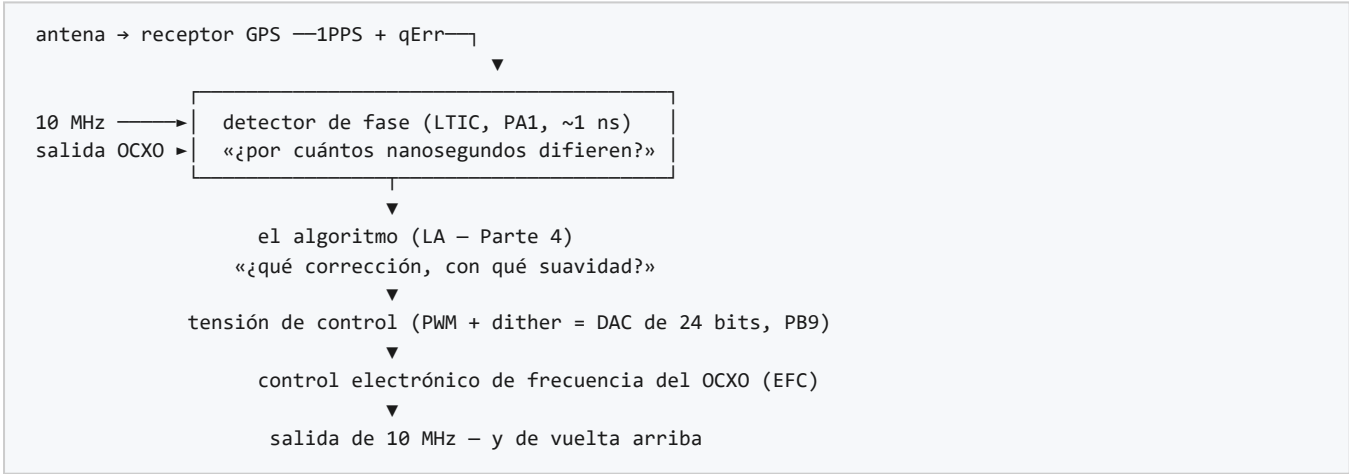
	Corto plazo (segundos)	Largo plazo (meses)
Solo el OCXO	excelente — uno bueno se mantiene en partes de 10^{11}	deriva — el envejecimiento y la temperatura lo desvían despacio
Solo el 1PPS del GPS	ruidoso — cada pulso cae a ~±25 ns de la verdad	excelente — gobernado por relojes atómicos, para siempre

La parte de «horno» del OCXO ya es media historia: el cristal vive en una cajita calentada a una temperatura constante (ese es el calentamiento de 300 segundos al arrancar), porque la temperatura es su mayor enemigo. Lo que queda es el envejecimiento — un paseo lento que nadie puede apagar.

El pulso del GPS es lo contrario: cada segundo individual es solo aproximado (el receptor cuantiza el tiempo, la señal se retrasa en la ionosfera, rebota en los edificios), pero el *promedio* de horas está clavado al tiempo atómico, porque los satélites cargan relojes atómicos y sin eso el GPS no funciona.

Un GPSDO es un reparto de trabajo: el OCXO se ocupa de los segundos, el GPS de los meses, y un lazo de control entre ambos mueve al primero, en pasitos diminutos, manteniéndolo anclado al segundo.

A.2 El lazo, pieza a pieza — y dónde está cada una en esta placa



- **El detector de fase responde «¿por cuánto nos retrasamos?»** una vez por segundo, en nanosegundos. Ese único número — la *fase* — es el marcador en el que se juega todo el partido.
- **El contador (TIM2) responde «¿vamos rápidos o lentos?»** contando literalmente los 10 MHz. Es grueso (pasos de 0,01 Hz) pero directo.
- **El algoritmo** mira esos números y decide una sola cosa: cuánto empujar la tensión de control, y en cuánto tiempo repartir el empujón.
- **El DAC**: PWM de 65536 pasos, con dither hasta unos efectivos 24 bits (pasos de ~0,2 µV — el oscilador nota menos de un paso PWM, así que el firmware recuerda la fracción entre pasos).
- **El pin EFC del OCXO**: voltios dentro, hercios fuera — todo el rango de ajuste mide apenas unos pocos hercios de ancho, y por eso los microvoltios importan.

A.3 Fase, frecuencia y la única multiplicación que hace falta

El error de frecuencia y la fuga de fase son el mismo hecho en dos disfraces. Si el oscilador se desvía en una fracción, su fase se desliza exactamente a ese ritmo:

Error de frecuencia a 10 MHz	La fase se desliza
1 Hz	100 ns por segundo
0,01 Hz	1 ns por segundo
0,001 Hz (1 mHz)	1 ns cada 10 segundos

Dos consecuencias dignas de grabar:

1. **Si la fase se detiene, las frecuencias son iguales** — sea cual sea el valor de la fase. La meta real del lazo es una fase *quieta*, aparcada cerca de cero.
2. Los errores que vale la pena perseguir son absurdamente pequeños. Un milihercio a 10 MHz es una parte en 10^{10} — y el lazo distingue rutinariamente diez veces mejor. Por eso cada cable, cada microvoltio y cada grado importan aquí más que en cualquier otro circuito que haya construido.

A.4 Por qué las correcciones deben ser suaves (el corazón del asunto)

Suponga que el pulso del GPS llega 20 ns tarde porque la señal rebotó en un edificio. El «error» de fase medido es 20 ns — pero no es real: el oscilador no se movió. Si el lazo corrige al instante a valor nominal, **copia el ruido del GPS sobre el oscilador** y la salida acaba *peor* que el OCXO libre. Ese único error es lo que separa un GPSDO de un oscilador apaleado por el GPS.

La defensa es promediar: el ruido aleatorio se reduce con √N. Promedie 100 segundos → ruido ÷10; 1000 s → ÷31. El lazo espera, pues, a que los números promediados del GPS queden más silenciosos que la propia deriva del oscilador, y solo entonces actúa — con una corrección a la medida de lo que el promedio largo realmente demostró, repartida en un tiempo comparable. Por eso:

- el algoritmo 11 tiene `LTC` (una constante de tiempo — «cuánta paciencia tiene este lazo»),
- el algoritmo 12 deja que **el tamaño del error elija su propio tiempo de promediación** (un error grande queda demostrado en segundos y se corrige en segundos; uno diminuto, a lo largo de una hora y a lo largo de una hora),
- cada corrección de este firmware se reparte en pasos sub-LSB en vez de volcarse de golpe.

Un almuerzo gratis: **la corrección de diente de sierra (`SAW 1`)**. El receptor *sabe* cuánto cuantizó cada pulso (el número `qErr`) y lo confiesa cada segundo. Restar esa confesión convierte ±25 ns de error falso en unos pocos ns antes de que empiece promedio alguno.

A.5 Holdover

Cuando el GPS desaparece (antena cortada, receptor muerto), al lazo le falta la verdad con la que gobernar. La jugada correcta es **congelar** la última tensión de control buena y dejar que el OCXO navegue con su propia estabilidad — eso es el holdover (`MH` manual, o automático cuando se pierde el fix o falta el 1PPS durante 10 s; el LED amarillo parpadea, la pantalla se pone naranja, `[HOLDOVER]` sustituye a la tendencia en el informe). El oscilador derivará entonces con el envejecimiento y la temperatura — despacio, pero imparable, hasta que el GPS vuelva. Desde el build 68 se comprueba el pulso además del fix, porque un receptor de tiempo en Time Mode da su posición fija como válida sin señal alguna; y el holdover automático termina sólo cuando el fix vuelve con 10 s de pulsos sin interrupción, así que una antena que va y viene no lo termina con uno o dos pulsos. Un holdover manual sigue siendo manual aunque se pierda así la referencia (build 68): la pérdida se anuncia (`GPS fix lost — manual holdover kept`, o `GPS 1PPS lost — …`), y terminarlo le toca a usted, con `MD`, no a la vuelta del GPS.

A.6 Cómo se juzgan los resultados: la desviación de Allan en un párrafo

Los patrones de frecuencia se comparan con la **desviación de Allan (ADEV)**: más o menos «cuánto discrepa el promedio consigo mismo, tomado sobre τ segundos», graficada contra τ. La curva de un GPSDO cae de izquierda a derecha: en τ = 1 s se ve el ruido propio del oscilador; al crecer τ, el promedio vence al ruido y el anclaje del GPS toma el mando (en el

banco del autor: ~$3 \cdot 10^{-9}$ a 1 s cayendo hasta ~$5 \cdot 10^{-12}$ a 3000 s). Cuando alguien publica «1e-12 en tau 10 000», esa es la frase que dice. Un *joroba* en medio de la curva significa que el lazo pelea consigo mismo — demasiado rápido para su propio ruido — y es la firma clásica de un GPSDO mal ajustado.

Apéndice B — PID para reacios

Nunca *necesita* este apéndice para usar el aparato — `CT` ajusta los lazos por usted. Es para el día en que abra la pestaña **PID algo 3-9** del sintonizador, o escriba `KP`, y quiera saber qué palanca sostiene y hacia dónde muerde. No se usa matemática más allá de la multiplicación.

B.1 La imagen de la ducha

Todo lazo de control jamás construido hace las mismas cuatro cosas:

1. **Medir** dónde está (la temperatura del agua).
2. **Comparar** con dónde debería estar (lo cómodo).
3. La diferencia es el **error** (frío por 5 grados).
4. **Actuar** (abrir el grifo del caliente), esperar, repetir.

La única pregunta de toda la materia es el paso 4: *¿cuánto abrir el grifo?* PID — tres letras, tres respuestas — es la receta estándar.

B.2 P — proporcional: «reacciona al ahora»

Abra el grifo en proporción al error. Frío por mucho → abra mucho. Frío por poco → abra poco.

- **P a solas tiene un defecto:** necesita algo de error para producir acción alguna, así que se asienta *cerca* del objetivo, nunca en él — el último grado de frío no alcanza para mantener el grifo abierto. Ese residuo se llama **droop**.
- **Demasiada P:** se pasa de caliente, luego corrige de más al frío, y oscila entre ambos — la ducha del infierno.
- **Muy poca P:** tarda una eternidad en llegar.

B.3 I — integral: «recuerda el pasado»

Mire el error *en el tiempo*. Aun un frío diminuto y persistente se acumula, en una suma corriente, hasta que el lazo añade acción suficiente para cerrarlo. **I es lo que mata el droop** — es el término «al final, exactamente».

- **Demasiada I:** el clásico yoyó lento. La suma crece durante la aproximación, luego se gasta como rebasamiento, se reconstruye al otro lado, y el lazo se mece con calma por una eternidad.
- **Windup:** si el grifo ya está abierto del todo (la salida en su límite), la suma sigue creciendo inútilmente y luego tarda siglos en deshacerse. El remedio es una pinza en la suma — y exactamente eso es `IL` (I_LIMIT).

B.4 D — derivativo: «anticipa»

Reaccione no al error sino a lo *rápido* que cambia. ¿Se caldea deprisa? Empiece a cerrar el grifo *antes* de llegar. **D es el amortiguador** — la suspensión que corta las oscilaciones que P e I adorarían.

- **El vicio de D:** amplifica el ruido de medición (derivar un sensor tembloroso produce agujas). Por eso los términos D suelen ir filtrados, pequeños, o ambas cosas — y por eso una medición de fase ruidosa empeora un ajuste cargado de D en vez de mejorarlo.
- En los algoritmos PLL (4/5/7) la ranura `Kd` actúa sobre la *fase* acumulada y no sobre una derivada cruda — el mismo oficio de amortiguar, con otro cableado. No deje que la letra despiste; piense «la perilla de la amortiguación».

B.5 Las tres letras, una tabla

Perilla	Reacciona a	Cura	Demasiada →	Muy poca →
P	el error de ahora mismo	la desgana	rebasamiento, oscilación rápida	eternidad en llegar
I	el error acumulado en el tiempo	el offset permanente (droop)	yoyó lento, windup	se asienta cerca, nunca en el objetivo

Perilla	Reacciona a	Cura	Demasiada →	Muy poca →
D	la velocidad de cambio del error	el rebasamiento (amortiguación)	nervios persiguiendo ruido	el rebasamiento resuena

B.6 Dónde están las perillas en este firmware

Lo que gira	Dónde	Qué letra es en realidad
`KP n val` / `KI` / `KD` / `IL n val`	algoritmos 3–9 (pestaña **PID algo 3-9**)	literalmente P, I, D y la pinza del integrador
`AQP…AQL`, `DPP…DPL`, `LKP…LKL`	las tres etapas del algoritmo 10 (pestaña **LTIC**)	un juego completo de PID por etapa — el lazo se reajusta solo mientras se asienta
`LG` / `LD` / `LTC`	algoritmo 11 (pestaña **LTIC-Lars**)	ganancia (con cuánta fuerza), amortiguación (cómo se asienta), constante de tiempo (con cuánta paciencia) — las mismas tres perillas con la ropa de Lars
`MG` y los límites	algoritmo 12 (pestaña **Multi-level**)	sin PID — véase abajo

El algoritmo 12 merece un párrafo honesto: **no tiene P, ni I, ni D**. En vez de ganancias fijas pregunta, cada vez, «¿qué tan grande es el error, cuánto promedié para demostrarlo?» y escala la corrección al par — los errores grandes reciben trato rápido y firme; los pequeños, trato lento y suave. Es la misma física con el ajuste incorporado, y por eso sus únicas perillas manuales son la ganancia (`MG`, y `CT` la mide por usted) y los límites que deciden qué cuenta como «grande».

B.7 Por qué existe `CT` — el problema de la regla

Los números del PID viven en unidades de «pasos PWM por hercio de error». Pero un paso PWM vale *una cantidad distinta de hercios en cada oscilador individual* — 0,32 mHz en uno de los Vectron del autor, casi siete veces menos en una construcción de EFC estrecho. Sin medir su oscilador, el mismo `Kp = 1000` es un empujoncito suave en una placa y un empujón violento en otra. **`CT` mide la respuesta voltios-a-hercios de su oscilador y reescala cada ganancia a juego** — para que el ajuste de fábrica signifique lo mismo físico en cada unidad. Por eso el orden de calibración de este manual es ley: primero `CT`, después `LC`, el ajuste (si alguna vez) al final.

B.8 Diez reglas de pulgar

1. Ejecute `CT` y `LC` antes de tocar ganancia alguna. En la mayoría de las vidas, ahí termina el ajuste.
2. Cambie **una** perilla a la vez.
3. Duplique o parta por dos — nunca ×10. El ajuste de lazos reacciona a proporciones, no a aritmética.
4. Dele una hora a cada cambio. Estos lazos promedian por minutos; juzgar un cambio a los treinta segundos es leer un libro por una letra.
5. Ante la duda, vaya **más lento** (más `LTC`, menos ganancia). Lento es solo aburrido; rápido es inestable.
6. Oscilación → recorte P primero, luego I.
7. Un offset que nunca termina de cerrar → más I — o, más probable, se saltó `CT`.
8. La salida persiguiendo visiblemente el ruido del GPS → lazo más lento, `SAW 1`, mejor cielo para la antena. No más amortiguación.
9. `ES` tras cada sesión; `ER` des-experimenta una mala tarde.
10. Si le pelea un día entero, sospeche del hardware antes que del ajuste: vista del cielo de la antena, temperatura (radiadores, puertas, sol), cableado EFC. A los lazos de este firmware es difícil romperlos y fácil culparlos.

Apéndice C — Glosario

Términos que aparecen por todo este manual, la telemetría y el changelog, en el sentido en que los usa este proyecto. Sugerencia de Alan Cashin, que señaló que la mitad de ellos se usan de otro modo en otras partes.

Término	Qué significa aquí
ADEV	Desviación de Allan — la medida estándar de estabilidad de un oscilador: cuánto cambia la frecuencia fraccional entre ventanas de promediado contiguas de longitud τ. Se lee como «a 100 s, este reloj llega a 1e-11».
ACQ / DPLL / LOCK	Las tres etapas del algoritmo 10. ACQ acerca la frecuencia usando el contador; DPLL asienta fase y frecuencia deprisa; LOCK corrige despacio y en banda estrecha para acercarse al error mínimo.
BBR	RAM respaldada por batería en un módulo GNSS — donde una configuración guardada del receptor sobrevive a un corte de alimentación, si hay pila de respaldo.
DAC	Convertidor digital-analógico: lo que convierte un número en la tensión de control. En este montaje normalmente no es un chip — véase PWM y dither.
código DAC / LSB	El número que se escribe en la salida de tensión de control, y un paso suyo. Todas las ganancias se expresan por LSB, porque es el movimiento más pequeño que el lazo puede hacer.
dither	Variar a propósito los bits bajos del ciclo de trabajo del PWM de periodo en periodo para que el *promedio* caiga entre dos pasos del hardware. 13 bits físicos reproducidos sobre una tabla de 2048 entradas dan unos 24 bits efectivos.
EFC	Control electrónico de frecuencia — la entrada de ajuste del OCXO. Voltios dentro, hercios fuera.
holdover	Funcionar sin GPS: el lazo deja de corregir y el OCXO corre libre con su última tensión de control.
LTIC	El contador de intervalos de tiempo de Lars — el detector de fase por rampa en PA1 (un condensador cargado entre el flanco del PPS y el del OCXO dividido). «TIC» a secas es el mismo detector.
NMEA	Las frases de texto que envía un módulo GNSS (`$GPRMC,…`): posición, hora, número de satélites — todo salvo la configuración binaria.
OCXO	Oscilador de cristal con horno: un cristal mantenido a temperatura constante, por eso es estable y por eso necesita calentamiento.
picDIV	Un divisor pequeño que convierte 10 MHz en un flanco de 1 Hz para el detector de fase. Hay que *armarlo* (resincronizarlo) para que su flanco caiga cerca del PPS.
PID / PI	El regulador: **P** reacciona al error de ahora, **I** al error acumulado, **D** a lo rápido que cambia. Casi todos los lazos aquí son PI — véase el Apéndice B.
PPS	La salida de un pulso por segundo del receptor GNSS — la referencia de tiempo contra la que se mide todo.
PWM	Modulación por ancho de pulso: una onda cuadrada cuyo ciclo de trabajo, tras un filtro RC, se convierte en la tensión de control. Barato y, con dither, mejor que la mayoría de los chips DAC.
qErr / diente de sierra	La propia estimación del receptor, en picosegundos, de cuánto falló su flanco de PPS respecto al tiempo verdadero. Los receptores de tiempo la informan (`UBX-TIM-TP`); corregirla elimina un error con forma de diente de sierra.
survey-in	Un receptor de tiempo midiendo su propia posición durante mucho rato para poder luego fijarla y dedicar todo su cálculo al *tiempo*.
Time Mode	El estado en el que entra un receptor de tiempo tras el survey-in: posición fija, tiempo optimizado. La pantalla muestra `HDOP:TIME`.
trend	La palabra de cuatro caracteres en la telemetría y en la pantalla que nombra lo que el lazo está haciendo ahora mismo: `ACQ`, `DPLL`, `LOCK`, `CORR`, `ZC`, `NOPH`, …
Vctl / Vphase	Vctl es la tensión de control que va *hacia* el oscilador; Vphase es la tensión del detector que vuelve *desde* la medida de fase. Dos pines distintos, fáciles de confundir.
ZC	Cancelación en el cruce por cero (algoritmo 12): retirar un empuje deliberado justo cuando la fase cruza cero, dejando frecuencia y fase correctas a la vez.

Apéndice D — El filtro de Kalman en palabras llanas

La parte 4.6 describe el algoritmo 13 por sus perillas; este apéndice lo describe por su intuición. Sin fórmulas — unas imágenes y ya.

D.1 Tres creencias, y un lápiz para cada una

El filtro lleva dentro tres creencias: **a cuánta distancia está la fase del oscilador** (en nanosegundos), **con qué rapidez crece ese desfase** (picosegundos por segundo) y **cómo cambia ese derivo con la edad** (envejecimiento). Con cada creencia lleva también **el grosor de un lápiz**: el intervalo del que sabe que no sabe. «Fase = 2 ns ± 1 ns» significa: seguro a un nanosegundo — y a lo largo de la noche esa certeza crece y mengua sola.

Los lápices son la mitad del filtro. Cuando llega una medición, la creencia se mueve **hacia ella, pero sólo hasta donde el lápiz de la medición permite contra el lápiz de la propia creencia**. Medición segura contra creencia difusa — un paso grande. Medición difusa contra creencia segura — un toque ligero. Eso es todo el «filtro de Kalman»: creencia con pesos, renovada cada segundo.

D.2 Dos testigos

- El **detector de fase** es fresco pero parlanchín: cada segundo informa de la fase con ~2,5 ns de ruido, y su cero deriva por su cuenta (~2,6 ns en ~45 s). El filtro mide ese ruido él mismo — con las diferencias entre lecturas sucesivas — y lo llama R.
- El **contador TIM2** es honesto pero tosco: habla de frecuencia con una precisión que sólo se vuelve útil al cabo de cien segundos. Pero sólo calla cuando todo calla — y es lo que mantiene vivo el lazo cuando el detector se queda ciego.

En corto: en los segundos manda el oscilador, en los meses manda el GPS, y el filtro **elige las proporciones de nuevo cada segundo** — en vez de una vez y para siempre, como la constante de tiempo del algoritmo 11.

D.3 El entendimiento y las manos: KT y KC

Desde el build 36 son dos perillas separadas, y la diferencia merece la pena:

- **KT** es la paciencia del **entendimiento**: cuán desparramada lleva la creencia en el tiempo y cuán rápido se le permite concluir (también pone la cota de Q — el filtro no puede correr más rápido que su horizonte).
- **KC** es la velocidad de las **manos**: dado que el filtro ya *sabe* que la fase está 5 ns demasiado lejos — ¿en cuánto tiempo se cierra eso? Por defecto KT/3 (KT/6 en los builds 61–67), pero sólo **tras el primer bloqueo**: en arranque en frío el lazo camina hasta la banda al paso tranquilo de KT (como mucho 100 s desde el build 67), y las manos rápidas se activan solas cuando ya hay algo rápido que cerrar. Manos más rápidas significan menos error de fase *registrado* (ADEV ~23% mejor en tau medio) pero más dither en el DAC — un presupuesto, no un almuerzo gratis.
- **KF** son las manos **con prisa**: cuando algo ha sacado la fase bien fuera — una caída de la alimentación, un salto del cristal — la cierran en 16 s hasta que lleva dos minutos en casa, y vuelven a KC (build 69). El funcionamiento en calma no lo ve.

D.4 Escepticismo sano, o qué hace el filtro cuando algo va mal

- **La puerta 4σ.** Una lectura demasiado salvaje para las propias predicciones del filtro se rechaza — en singles o parejas en eventos GPS es lo normal (la noche del build 42: 320 rechazos en 20 horas, la racha más larga, dos). Diez seguidas y la puerta cede: lo erróneo es el estado, no los datos.
- **La prueba de confianza.** Fase y frecuencia son el mismo espectáculo visto desde dos lados: si el contador dice que se está transportando frecuencia, la fase *tiene* que moverse. Un detector que no se mueve cuando debía, miente. La prueba ignora titubeos menores que la resolución del propio contador (no puede condenar a un detector sano con el ruido de la propia referencia) y pregunta qué hace la frecuencia *ahora*: la media del contador va hasta cien segundos por detrás, así que el lazo le suma las correcciones que ha hecho desde entonces (build 65 — antes, un enganche podía condenar a un detector sano por el movimiento de cincuenta segundos atrás). Un detector condenado recupera su voz al cabo de media hora, o en el siguiente armado si su lectura salta con el divisor, cosa que uno congelado no puede hacer.

- **Silencio tras un arm.** Armar el divisor lo detiene un segundo, mientras la rampa sigue muestreándose y lee el raíl superior (~1400 ns — parece una fase, pero no lo es). Durante cuatro segundos tras un arm el filtro sencillamente no oye la palabra «fase». Cuatro segundos de `NOPH` justo después de un `ARM` en el log son señal de salud, no de avería.
- **Sin lectura no hay fase.** Tras un reinicio o un armado, y tras diez segundos sin lectura, el filtro no guarda una fase propia: gobierna solo con la frecuencia hasta que el detector vuelva a leer dentro de la banda. Hasta el build 65 llevaba una fase construida con su propia estimación de frecuencia y gobernaba para anularla; el 07.10, arrancando a 0,48 Hz con el detector en el raíl, eso llevó el oscilador 0,8 Hz hacia el otro lado. Sin fase, una lectura del contador rechazada diez veces seguidas se cree en vez de recortarse — sin nada más a qué atenerse, una lectura que insiste en discrepar tiene más probabilidades de acertar que el filtro.
- **Por debajo de la ventana.** La ventana del detector es ±0,55 del span de `LC` alrededor de su cero, y la pantalla se atiene a ella. El algoritmo 13 lee también por debajo, hasta 0,9 del span (nunca bajo 0,15 V): ahí la rampa RC es más empinada y sigue siendo unívoca, y ahí es donde aterriza el divisor tras un armado. Una lectura así se toma por quince veces más ruidosa que una en el cero (build 66 — antes, un aterrizaje por debajo de la ventana no era lectura alguna, y un rearme aterriza donde aterrizó el anterior).
- **Los bordes pesan menos.** Más allá de un cuarto de la banda desde el cero, una lectura se toma por más ruidosa cuanto más lejos está — diez veces en el borde de la banda (build 66): ahí el detector es más ruidoso y su lectura en línea recta menos fiel, y dos lecturas a 19 ns una de otra justo tras un aterrizaje pasaron una vez por una pendiente de 17 ns/s. Un lazo en lock no lo nota: empieza fuera de la banda que fija `LAT`.
- **Una lectura repetida.** Cuando el firmware no puede leer una ráfaga en su flanco, publica de nuevo la última lectura y la tendencia dice `KEPT`: el filtro no toma fase de ella, pero tampoco es un hueco. En lock, un segundo de cada mil; durante un enganche, uno de cada tres es normal.
- **Silencio del TIM2 tras un reinicio.** Durante los primeros cien segundos la media de cien del contador aún lleva el calentamiento del oscilador; el filtro espera una medición limpia en vez de creer en la historia.

D.5 Cómo se ve una buena noche

Tendencia `KAL` durante ~99% del tiempo. `REJ` sueltos en eventos GPS. `NOPH` sólo los cuatro segundos tras un `ARM`, `KEPT` alrededor de un segundo de cada mil. R se mantiene en ~1,3 ns en la placa del autor (la noche del build 64; noches de banco más antiguas, ~2,9), `sig` ~1 ns. `[at ceiling]` junto a Q — en este banco los microeventos GPS empujan la adaptación toda la noche, así que Q se la pasa en la cota; propiedad del cielo, no avería. `KL` tras la noche añade el ratio de adaptación y las marcas de agua de Q (`lo..hi`) — de ahí se ve si Q salió alguna vez de la cota.

D.6 Cuándo no tocar nada

Los valores por defecto de KR/KQ/KT/KC están **medidos en la propia placa** — R de su detector, Q de su oscilador, el resto de los horizontes. Fijar valores (`KR` al suelo blanco, `KQ` en la semilla) es para experimentos comparativos, no para el trabajo diario: cada valor fijado le dice al filtro que sabe más que la placa — y la placa normalmente no miente. Si quiere ver *si* el filtro tiene razón, el mejor hábito sigue siendo: `KL`, `SW`, un log de una hora — y el Apéndice A, para saber qué mirar.